Пакет gtm_detector/ml_freq

Построчный разбор модулей

Данные, признаки, оценка качества, командная строка и тесты

ML-детекция ГТМ «увеличение частоты F»

26.10.2026

Содержание

Как устроен документ ... 3
Путь данных через пакет ... 3
Модули в этом документе ... 3
Пакет ml_freq: __init__.py и __main__.py ... 3
Файл gtm_detector/ml_freq/__init__.py ... 3
Файл gtm_detector/ml_freq/__main__.py ... 4
config.py — все настройки в одном месте ... 4
Файл gtm_detector/ml_freq/config.py ... 4
data/models.py — модели данных ... 7
Файл gtm_detector/ml_freq/data/models.py ... 7
data/readers.py — чтение Excel технологов ... 9
Файл gtm_detector/ml_freq/data/readers.py ... 9
Файл gtm_detector/ml_freq/data/__init__.py ... 14
features/builder.py — признаки по дням ... 15
Файл gtm_detector/ml_freq/features/builder.py ... 15
Файл gtm_detector/ml_freq/features/__init__.py ... 19
evaluation/decoding.py — вероятности → мероприятия ... 19
Файл gtm_detector/ml_freq/evaluation/decoding.py ... 19
evaluation/metrics.py — сравнение с разметкой и метрики ... 20
Файл gtm_detector/ml_freq/evaluation/metrics.py ... 20
evaluation/experiment.py — перекрёстная проверка ... 23
Файл gtm_detector/ml_freq/evaluation/experiment.py ... 23
evaluation/report.py — отчёт: Excel и график ... 25
Файл gtm_detector/ml_freq/evaluation/report.py ... 25
Файл gtm_detector/ml_freq/evaluation/__init__.py ... 27
cli.py — командная строка ... 27
Файл gtm_detector/ml_freq/cli.py ... 27
tests/conftest.py — синтетические скважины для тестов ... 30
Файл gtm_detector/ml_freq/tests/conftest.py ... 30
tests/test_models_and_evaluation.py — тесты моделей и оценки ... 32
Файл gtm_detector/ml_freq/tests/test_models_and_evaluation.py ... 32

Как устроен документ

Здесь разобраны все модули пакета `gtm_detector/ml_freq`, кроме уже разобранных раньше: папки `models/` (`base.py`, `classic.py`, `transformer.py`, `__init__.py`) и теста `tests/test_data_and_features.py`.

Для каждого модуля: зачем он нужен, затем фрагменты кода с номерами строк (как в редакторе) и объяснение каждого фрагмента. Код вставлен из файлов автоматически, поэтому совпадает с ними дословно.

Путь данных через пакет

```
Excel (замеры + разметка)
   │  data/readers.py      MeasurementsReader, LabelsReader, WellRepository (+ кэш)
   ▼
WellSeries (ряды по дням + размеченные Interval)
   │  features/builder.py  FeatureBuilder: sequence() / tabular()
   ▼
признаки по дням
   │  models/              Transformer или модель scikit-learn: fit / predict_proba
   ▼
вероятность эффекта для каждого дня
   │  evaluation/decoding.py   SegmentDecoder
   ▼
мероприятия (начало, окончание)
   │  evaluation/metrics.py    EventMatcher, Score  — сравнение с разметкой
   │  evaluation/experiment.py CrossValidator       — k-fold по скважинам
   ▼  evaluation/report.py     ComparisonReport     — comparison.xlsx + .png
```

Модули в этом документе

Файл	Назначение
`__init__.py`, `__main__.py`	пакет и запуск `python -m gtm_detector.ml_freq`
`config.py`	все настройки: пути, декодер, архитектура сети, обучение, проверка
`data/models.py`	`Interval` (мероприятие), `WellSeries` (скважина), разметка и маска обучения
`data/readers.py`	чтение Excel, нормализация номеров, разбор дат с опечатками, кэш
`features/builder.py`	15 каналов по дням и 106 табличных признаков
`evaluation/decoding.py`	вероятности по дням → мероприятия
`evaluation/metrics.py`	сопоставление с разметкой, полнота, точность, ошибки дат
`evaluation/experiment.py`	перекрёстная проверка по скважинам
`evaluation/report.py`	отчёт: Excel и график
`cli.py`	команды `compare`, `train`, `predict`
`tests/conftest.py`, `tests/test_models_and_evaluation.py`	синтетические скважины и тесты моделей и оценки

Пакет ml_freq: __init__.py и __main__.py

Файл gtm_detector/ml_freq/__init__.py

`__init__.py` превращает папку `ml_freq` в пакет Python: без него `import gtm_detector.ml_freq` не сработает. `__main__.py` — то, что выполняется по команде `python -m gtm_detector.ml_freq`.

```
"""ML-детекция мероприятий «увеличение частоты F».

Собственная модель — Transformer-энкодер для суточных рядов (PyTorch) — и готовые
модели scikit-learn решают одну задачу: для каждого дня вероятность того, что идёт
эффект мероприятия. Даты начала и окончания из вероятностей получает общий
декодер, поэтому модели сравниваются одинаково.
"""
```

Весь файл — одна строка документации (docstring) пакета. Кода нет: пакет ничего не импортирует при загрузке, поэтому `import gtm_detector.ml_freq` быстрый — PyTorch и scikit-learn подгружаются только тогда, когда нужны конкретные модули.

Текст описывает главную идею: и своя модель (Transformer), и готовые (scikit-learn) решают одну задачу — вероятность эффекта по каждому дню, а даты начала и окончания получает общий декодер. Поэтому модели сравниваются на равных.

Файл gtm_detector/ml_freq/__main__.py

```
import sys

from .cli import main

sys.exit(main())
```

Строка 1. `sys` нужен для `sys.exit`.

Строка 3. Импорт функции `main` из соседнего `cli.py`. Точка в `.cli` — относительный импорт: «из этого же пакета».

Строка 5. `main()` разбирает аргументы командной строки, выполняет команду и возвращает код завершения: 0 — успех, 1 — ошибка. `sys.exit(код)` передаёт его операционной системе — так скрипты и планировщики понимают, удалась ли команда.

Когда вы пишете `python -m gtm_detector.ml_freq compare`, Python находит пакет и выполняет именно этот файл.

config.py — все настройки в одном месте

Файл gtm_detector/ml_freq/config.py

Все числа, которые можно менять (пути, пороги, размеры сети, параметры обучения), собраны здесь, а не разбросаны по коду. Каждая группа — отдельный dataclass.

```
"""Конфигурация ML-детектора мероприятий «увеличение частоты F»."""

from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path
```

Строка 1 — описание модуля.

Строка 3 — `from __future__ import annotations`: аннотации типов хранятся как строки и не вычисляются при загрузке. Даёт современную запись `int | None` и ссылки на ещё не объявленные классы.

Строка 5 — `dataclass` автоматически создаёт для класса конструктор, сравнение `==` и красивый вывод по списку полей. `field` нужен для полей со сложным значением по умолчанию (строка 72).

Строка 6 — `Path` из `pathlib`: объектный путь к файлу. Склеивается оператором `/` и одинаково работает на Windows и Linux.

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
PACKAGE_DIR = Path(__file__).resolve().parent
```

Два пути, вычисляемые от расположения самого файла `config.py`:

- `Path(__file__)` — путь к `config.py`; `.resolve()` делает его абсолютным;

- `.parents[2]` — подняться на 3 уровня: `config.py` → `ml_freq` → `gtm_detector` → **корень проекта**. Индекс 0 — это сама папка `ml_freq`, 1 — `gtm_detector`, 2 — корень;
- `PACKAGE_DIR` — папка `ml_freq` (`.parent` — на уровень выше файла).

Почему не `os.getcwd()`: текущая папка зависит от того, откуда запущена программа. Путь от `__file__` всегда один и тот же, из какой бы папки вы ни запускали.

```
12  @dataclass(frozen=True)
13  class DataConfig:
14      """Где лежат данные и куда писать результаты."""
15
16      measurements: Path = PROJECT_ROOT / "resources" / "input_dataset.xlsx"
17      results: Path = PROJECT_ROOT / "resources" / "output_dataset.xlsm"
18      output_dir: Path = PROJECT_ROOT / "results" / "ml_freq"
19      weights_dir: Path = PACKAGE_DIR / "weights"
```

`DataConfig` — где данные и куда писать результаты.

`@dataclass(frozen=True)` — «замороженный» класс: после создания поля менять нельзя (`cfg.measurements = ...` вызовет ошибку). Настройки не меняются случайно посреди работы, а объект можно безопасно передавать куда угодно.

- `measurements` — файл замеров `resources/input_dataset.xlsx`;
- `results` — файл разметки `resources/output_dataset.xlsm`;
- `output_dir` — куда писать отчёты: `results/ml_freq`;
- `weights_dir` — куда сохранять обученные модели: `gtm_detector/ml_freq/weights`.

Значения по умолчанию можно переопределить при создании: `DataConfig(measurements=Path("другой.xlsx"))` — так делает командная строка с ключом `--measurements`.

```
22  @dataclass(frozen=True)
23  class DecoderConfig:
24      """Как превратить вероятности по дням в мероприятия."""
25
26      threshold: float = 0.5  # день «в эффекте», если сглаженная вероятность выше
27      smooth_days: int = 5  # окно скользящего среднего вероятностей
28      max_gap: int = 3  # разрывы короче — склеиваются в один период
29      min_length: int = 2  # более короткие периоды отбрасываются
```

`DecoderConfig` — правила превращения вероятностей по дням в мероприятия (их применяет `SegmentDecoder`):

- `threshold = 0.5` — день считается «в эффекте», если сглаженная вероятность выше 0.5;
- `smooth_days = 5` — вероятности сглаживаются средним за 5 дней, чтобы одиночный «всплеск» модели не дал отдельное мероприятие;
- `max_gap = 3` — если между двумя периодами «эффекта» разрыв до 3 дней, они склеиваются в один;
- `min_length = 2` — период короче 2 дней отбрасывается как шум.

```
32  # Дней до начала и после окончания размеченного мероприятия, на которых учатся модели.
33  # Остальные дни не используются: в разметке есть не все реальные подъёмы частоты.
34  CONTEXT_DAYS = 45
```

`CONTEXT_DAYS = 45` — обычная константа (не dataclass). Модели учатся только на днях от «начало − 45» до «окончание + 45» вокруг размеченных мероприятий. Причина в комментарии: в разметке есть не все реальные подъёмы частоты, поэтому день вне разметки нельзя считать «без эффекта». Используется в `WellSeries.training_mask` и в моделях.

```
37  @dataclass(frozen=True)
38  class TransformerConfig:
39      """Архитектура Transformer-энкодера для временных рядов."""
40
41      # Небольшая модель (~20 тыс. параметров): на ~100 скважинах крупнее — хуже
42      # (d_model=64, 3 слоя: точное начало 16% против 24% на перекрёстной проверке).
43      d_model: int = 32
44      n_heads: int = 4
45      n_layers: int = 2
46      dim_feedforward: int = 64
47      dropout: float = 0.2
48      conv_kernel: int = 5  # свёртка перед энкодером ловит локальные ступеньки
49      max_length: int = 4096
```

`TransformerConfig` — архитектура своей нейросети.

- `d_model = 32` — длина вектора, которым сеть описывает каждый день внутри себя;
- `n_heads = 4` — число «голов» внимания: 4 независимых способа связывать дни между собой. `d_model` должен делиться на `n_heads` (32 / 4 = 8 чисел на голову);
- `n_layers = 2` — сколько слоёв Transformer стоит друг за другом;
- `dim_feedforward = 64` — размер внутренней полносвязной сети в каждом слое;
- `dropout = 0.2` — при обучении случайно отключается 20% связей: защита от запоминания маленького набора данных;
- `conv_kernel = 5` — ширина окна свёртки в днях перед энкодером;
- `max_length = 4096` — максимальная длина ряда (дней), для которой заготовлено позиционное кодирование.

Комментарий в строках 41–42 фиксирует результат эксперимента: крупная модель (d_model 64, 3 слоя) на ~100 скважинах работала хуже — точное начало 16% против 24%.

```
52  @dataclass(frozen=True)
53  class TrainingConfig:
54      """Обучение Transformer."""
55
56      epochs: int = 80
57      batch_size: int = 16
58      learning_rate: float = 1e-3
59      weight_decay: float = 1e-4
60      crop_length: int = 128  # обучение на случайных окнах ряда — аугментация
61      crops_per_well: int = 4
62      seed: int = 0
```

`TrainingConfig` — параметры обучения Transformer.

- `epochs = 80` — сколько раз модель проходит по обучающим данным;
- `batch_size = 16` — сколько окон обрабатывается за один шаг обновления весов;
- `learning_rate = 1e-3` (0.001) — размер шага оптимизатора;
- `weight_decay = 1e-4` — регуляризация: веса понемногу тянутся к нулю;
- `crop_length = 128` — модель учится на случайных окнах по 128 дней, а не на целых рядах (аугментация данных);
- `crops_per_well = 4` — по 4 таких окна с каждой скважины за эпоху;
- `seed = 0` — зерно случайности: одинаковое зерно — одинаковый результат обучения.

```
65  @dataclass(frozen=True)
66  class ExperimentConfig:
67      """Перекрёстная проверка по скважинам."""
68
69      folds: int = 5
70      seed: int = 0
71      start_tolerance: int = 7  # дней: допуск при сопоставлении с разметкой
72      decoder: DecoderConfig = field(default_factory=DecoderConfig)
```

`ExperimentConfig` — перекрёстная проверка.

- `folds = 5` — скважины делятся на 5 частей; модель 5 раз учится на четырёх и проверяется на пятой;
- `seed = 0` — зерно для случайного разбиения скважин на части;
- `start_tolerance = 7` — найденное и размеченное мероприятие считаются одним, если даты начала отличаются не больше чем на 7 дней;
- `decoder` — настройки декодера (вложенный `DecoderConfig`).

Почему `field(default_factory=DecoderConfig)`, а не просто `= DecoderConfig()`: значение по умолчанию у поля dataclass вычисляется один раз и было бы общим для всех объектов. `default_factory` создаёт новый объект при каждом создании `ExperimentConfig`. Для замороженного класса это скорее аккуратность, но dataclass требует такой записи для изменяемых значений по умолчанию.

data/models.py — модели данных

Файл gtm_detector/ml_freq/data/models.py

Два класса, которыми оперирует весь модуль: `Interval` (одно мероприятие — период эффекта) и `WellSeries` (скважина с суточными рядами и разметкой).

```
"""Модели данных: скважина с суточными рядами и интервалы мероприятий."""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import date

import numpy as np
import pandas as pd
```

Импорты: `dataclass` и `field` — для классов данных, `date` — тип даты, NumPy — массивы, pandas — таблица рядов скважины.

```
SERIES_COLUMNS = (
    "frequency",
    "debit_liquid",
    "debit_oil",
    "watercut",
    "pressure_zab",
    "pressure_line",
)
```

`SERIES_COLUMNS` — названия суточных рядов скважины, как они называются в коде: частота, дебит жидкости, дебит нефти, обводнённость, забойное и линейное давление. Кортеж (а не список) — чтобы его нельзя было случайно изменить. Используется как справочник имён колонок.

```
@dataclass(frozen=True)
class Interval:
    """Мероприятие: период эффекта. ``end=None`` — эффект продолжается до конца данных."""

    start: date
    end: date | None
    date_done: date | None = None  # «Дата выполнения» (только в разметке)

    def covers(self, day: date, last_day: date) -> bool:
        return self.start <= day <= (self.end or last_day)
```

`Interval` — одно мероприятие, точнее период его эффекта. `frozen=True` — неизменяемый: интервалы можно сравнивать (`==`) и класть в множества.

- `start` — дата начала эффекта;
- `end` — дата окончания или `None`, если эффект продолжается до конца данных (так возвращает модель, когда «эффект» доходит до последнего дня);
- `date_done` — «Дата выполнения мероприятия» из разметки; у предсказаний модели её нет (по умолчанию `None`).

Метод `covers(day, last_day)` — попадает ли день в период. Если `end` пустой, вместо него берётся `last_day`: запись `self.end or last_day` вернёт `last_day`, когда `end` равен `None`. Двойное сравнение `a <= x <= b` в Python читается как «x между a и b».

Один класс используется и для разметки, и для предсказаний — так их легко сравнивать.

```
@dataclass
class WellSeries:
    """Скважина: суточные ряды на непрерывном календаре + разметка."""

    number: str
    cluster: str
    frame: pd.DataFrame  # индекс — даты, колонки — SERIES_COLUMNS и state
    labels: list[Interval] = field(default_factory=list)
```

`WellSeries` — скважина. Без `frozen`: в объект позже добавляется разметка (`labels`).

- `number` — номер скважины (строка: бывают буквы, например «539Б»);
- `cluster` — номер куста. Номера скважин на месторождении повторяются, поэтому скважину однозначно задаёт только пара (номер, куст);

- `frame` — таблица pandas: строки — дни подряд без пропусков, столбцы — ряды из `SERIES_COLUMNS` и `state` (1 — скважина работает, 0 — стоит);
- `labels` — список размеченных мероприятий. `field(default_factory=list)` — у каждой скважины свой пустой список. Если написать `= []`, все скважины делили бы **один** список — классическая ошибка в Python.

```
42      @property
43      def well_id(self) -> str:
44          return f"{self.number}/{self.cluster}"
45
46      @property
47      def dates(self) -> list[date]:
48          return [day.date() for day in self.frame.index]
49
50      def __len__(self) -> int:
51          return len(self.frame)
```

Вспомогательные свойства (`@property` — вызываются без скобок):

- `well_id` — текстовый идентификатор «номер/куст», например `539Б/792`;
- `dates` — список дат скважины в виде `date`. Индекс таблицы — `pandas.Timestamp`, `.date()` превращает каждый в обычную дату;
- `__len__` — «магический» метод: позволяет писать `len(well)` — число дней.

```
53      def target(self) -> np.ndarray:
54          """1 — день внутри размеченного периода эффекта, 0 — вне."""
55          days = self.frame.index
56          target = np.zeros(len(days), dtype=np.float32)
57          for label in self.labels:
58              end = pd.Timestamp(label.end) if label.end else days[-1]
59              target[(days >= pd.Timestamp(label.start)) & (days <= end)] = 1.0
60          return target
```

`target()` — правильные ответы для обучения: массив чисел по дням, 1.0 — день внутри размеченного периода эффекта, 0.0 — вне.

- Строка 55: `days` — индекс таблицы (даты).
- Строка 56: массив нулей длиной в число дней; тип `float32` — такой ждёт PyTorch.
- Строки 57–59: для каждого мероприятия ставим 1 во все дни от начала до окончания включительно. Если окончание пустое (`None`), берётся последний день ряда. `(days >= начало) & (days <= конец)` — булева маска дней внутри периода; присваивание `target[маска] = 1.0` записывает единицы сразу во все такие дни (векторная операция NumPy, без цикла по дням).

Если мероприятий несколько, их периоды просто объединяются.

```
62      def training_mask(self, context_days: int | None) -> np.ndarray:
63          """Дни, на которых можно учиться.
64
65          В разметке есть не все реальные подъёмы частоты, поэтому «вне разметки» не
66          значит «эффекта нет». Надёжны только окрестности размеченных мероприятий:
67          ``context_days`` дней до начала и после окончания. ``None`` — все дни.
68          """
69          days = self.frame.index
70          if context_days is None:
71              return np.ones(len(days), dtype=bool)
72          mask = np.zeros(len(days), dtype=bool)
73          margin = pd.Timedelta(days=context_days)
74          for label in self.labels:
75              end = pd.Timestamp(label.end) if label.end else days[-1]
76              mask[(days >= pd.Timestamp(label.start) - margin) & (days <= end + margin)] = True
77          return mask
```

`training_mask(context_days)` — на каких днях можно учиться. Возвращает массив True/False.

Строки 63–68 (docstring) объясняют зачем: в разметке есть не все реальные мероприятия, поэтому надёжны только окрестности размеченных.

- Строки 70–71: если `context_days` равен `None` — учиться на всех днях, массив из одних True.
- Строка 72: иначе начинаем со всех False.
- Строка 73: запас в виде интервала времени pandas (`Timedelta`), чтобы вычитать его из дат.

- Строки 74–76: для каждого мероприятия отмечаем True дни от «начало − запас» до «окончание + запас».

Пример: мероприятие 12.02–13.03, запас 45 дней → учимся на днях с 29.12 по 27.04. Скважины без разметки получают маску из одних False и в обучение не попадают.

data/readers.py — чтение Excel технологов

Файл gtm_detector/ml_freq/data/readers.py

Самый «грязный» модуль: он знает всё про устройство Excel-файлов (названия листов, номера столбцов, опечатки) и превращает их в аккуратные объекты `WellSeries`. Остальной код про Excel ничего не знает.

```
1  """Чтение Excel-файлов технологов.
2
3  * Замеры (``input_dataset.xlsx``): лист на параметр, строка — скважина,
4    столбцы ``ДД.ММ.ГГГГ`` — суточные значения; состояние — лист «Прим».
5  * Разметка (``output_dataset.xlsm``): лист «Результат__*».
6  """
```

Описание: что за файлы читаются и как они устроены — замеры (лист на параметр, строка — скважина, столбцы-даты) и разметка (лист «Результат__*»).

```
8  from __future__ import annotations
9
10 import hashlib
11 import logging
12 import pickle
13 import re
14 import tempfile
15 from collections.abc import Iterable
16 from datetime import date
17 from pathlib import Path
18 from typing import Any
19
20 import pandas as pd
21
22 from .models import Interval, WellSeries
23
24 logger = logging.getLogger(__name__)
```

Импорты стандартной библиотеки:

- `hashlib` — для «отпечатка» файлов в имени кэша;
- `logging` — сообщения о ходе работы;
- `pickle` — сохранение готовых объектов Python в файл (кэш);
- `re` — регулярные выражения для поиска дат в тексте;
- `tempfile` — путь к системной временной папке;
- `Iterable`, `date`, `Path`, `Any` — для аннотаций.

`logger = logging.getLogger(__name__)` — логгер с именем модуля; сообщения видны при запуске с ключом `-v`.

```
26 _LATIN_TO_CYRILLIC = str.maketrans("ABEKMHOPCTXabekmhopctx", "АВЕКМНОРСТХавекмнорстх")
27 _DATE_IN_TEXT = re.compile(r"(\d{1,2})\.(\d{1,2})\.(\d{2,4})")
28 _DATE_HEADER = re.compile(r"\d{2}\.\d{2}\.\d{4}")
```

Три заранее подготовленных объекта (подчёркивание в начале имени — «внутреннее, не для внешнего использования»):

- `_LATIN_TO_CYRILLIC` — таблица замены для `str.translate`: латинские буквы, внешне неотличимые от русских (A, B, E, K, M, H, O, P, C, T, X), заменяются русскими. В файлах номер «561E» может быть набран латиницей в одном месте и кириллицей в другом;
- `_DATE_IN_TEXT` — шаблон «1–2 цифры . 1–2 цифры . 2–4 цифры» для поиска даты внутри любого текста. Скобки `( )` — группы, которые потом извлекаются как день, месяц, год;
- `_DATE_HEADER` — строгий шаблон `ДД.ММ.ГГГГ` для распознавания столбцов-дат в шапке.

`re.compile` компилирует шаблон один раз при загрузке модуля, а не при каждом вызове.

```
31 def normalize_id(value: Any) -> str:
32     """Номер скважины/куста к единому виду: ``'561E '`` -> ``'561E'``, ``7782.0`` -> ``'7782'``."""
33     if value is None or (isinstance(value, float) and pd.isna(value)):
34         return ""
35     if isinstance(value, float) and value.is_integer():
36         value = int(value)
37     return str(value).strip().upper().translate(_LATIN_TO_CYRILLIC)
```

`normalize_id(value)` — номер скважины или куста к единому виду.

- Строки 33–34: пустое значение (`None` или NaN — так pandas обозначает пустую ячейку) → пустая строка.
- Строки 35–36: Excel часто отдаёт номер как дробное число `7782.0`; если дробная часть нулевая, превращаем в целое `7782`, иначе получилась бы строка «7782.0».
- Строка 37: в строку → убрать пробелы по краям (`strip`) → заглавные буквы (`upper`) → латиница в кириллицу (`translate`). Результат: `" 561e "` → `"561E"`.

Без этой функции разметка не находила бы свои скважины в файле замеров.

```
40 def parse_date(value: Any) -> date | None:
41     """Дата из ячейки, в том числе с опечатками: ``'21.04.2026г'``, ``'08.02.206'``."""
42     if value is None or (isinstance(value, float) and pd.isna(value)):
43         return None
44     if isinstance(value, (pd.Timestamp, date)):
45         timestamp = pd.Timestamp(value)
46     else:
47         found = _DATE_IN_TEXT.search(str(value))
48         if not found:
49             return None
50         day, month, year = map(int, found.groups())
51         year = {206: 2026, 26: 2026}.get(year, year)
52         try:
53             timestamp = pd.Timestamp(year=year, month=month, day=day)
54         except ValueError:
55             return None
56     if pd.isna(timestamp) or timestamp.year < 2000:
57         return None
58     return timestamp.date()
```

`parse_date(value)` — дата из ячейки, в том числе с опечатками. Возвращает `date` или `None`.

- Строки 42–43: пустая ячейка → `None`.
- Строки 44–45: если Excel уже отдал дату (тип `Timestamp` или `date`) — просто приводим к `Timestamp`.
- Строки 47–49: иначе это текст. Ищем в нём шаблон даты; не нашли — `None`.
- Строка 50: извлекаем три группы (день, месяц, год) и превращаем в числа. `map(int, ...)` применяет `int` к каждой.
- Строка 51: исправление опечаток в годе: 206 и 26 → 2026. Словарь `.get(year, year)` возвращает исправление, если оно есть, иначе сам год. Такие опечатки реально были в файле разметки («08.02.206»).
- Строки 52–55: собираем дату; если она невозможна (например, 31.02) — `ValueError`, возвращаем `None`.
- Строки 56–57: отсев мусора: пустая дата (`NaT`) или год раньше 2000.
- Строка 58: результат как обычная `date`.

Хвосты вроде «г» в «21.04.2026г» игнорируются автоматически: шаблон ищет дату внутри строки.

```
61 class MeasurementsReader:
62     """Файл замеров -> список :class:`WellSeries` (без разметки)."""
63 
64     SHEETS = {
65         "debit_liquid": "Qж",
66         "watercut": "Обв",
67         "debit_oil": "Qн",
68         "pressure_line": "Рлин",
69         "frequency": "Fэцн ТМ",
70         "pressure_zab": "Рзаб",
71     }
72     STATE_SHEET = "Прим"
73     WORKING_STATES = frozenset({"раб.", "нак"})
74 
75     def __init__(self, path: Path) -> None:
76         self.path = Path(path)
```

Класс `MeasurementsReader` — файл замеров → список скважин.

- `SHEETS` — какой лист Excel какому ряду соответствует: «Qж» → `debit_liquid` и т.д. Если в новом файле лист называется иначе, достаточно поправить словарь;
- `STATE_SHEET = "Прим"` — лист с состоянием скважин по дням;
- `WORKING_STATES` — какие состояния считаются «скважина работает»: «раб.» и «нак» (работа в режиме накопления). Остальные (ост., б/д, пьез, конс, набл) — «не работает». `frozenset` — неизменяемое множество, быстрая проверка «входит ли»;
- конструктор только запоминает путь к файлу; читается файл в `read()`.

```
78      def read(self) -> list[WellSeries]:
79          tables = {name: self._read_parameter(sheet) for name, sheet in self.SHEETS.items()}
80          states = self._read_states()
81          frequency = tables["frequency"]
82          calendar = pd.date_range(frequency.columns.min(), frequency.columns.max(), freq="D")
83
84          wells = []
85          for key in frequency.index:
86              frame = pd.DataFrame(index=calendar)
87              for name, table in tables.items():
88                  frame[name] = table.loc[key].reindex(calendar) if key in table.index else float("nan")
89              if key in states.index:
90                  frame["state"] = states.loc[key].reindex(calendar).isin(self.WORKING_STATES).to_numpy()
91              else:
92                  frame["state"] = (frame["frequency"] > 0).to_numpy()
93              wells.append(WellSeries(number=key[0], cluster=key[1], frame=frame.astype(float)))
94          logger.info("Замеры: %d скважин, %s — %s", len(wells), calendar[0].date(), calendar[-1].date())
95          return wells
```

`read()` — главный метод.

- Строка 79: читаем все шесть листов параметров. Получаем словарь «имя ряда → таблица (скважины × даты)».
- Строка 80: читаем лист состояний.
- Строки 81–82: строим **непрерывный календарь** от первой до последней даты листа частоты, с шагом в 1 день. Даже если в каких-то днях данных нет, в календаре они будут — тогда «шаг на одну строку» всегда равен одному дню, что важно для признаков и модели.
- Строки 85–88: для каждой скважины (ключ — пара номер/куст) создаём таблицу на календаре и заполняем столбцы рядами. `reindex(calendar)` раскладывает значения по датам календаря, отсутствующие дни становятся NaN. Если скважины нет на каком-то листе — весь столбец NaN.
- Строки 89–92: состояние. Если скважина есть на листе «Прим» — True там, где состояние «раб.» или «нак» (`isin`). Если нет — считаем, что работала там, где частота больше нуля.
- Строка 93: создаём `WellSeries`; `astype(float)` переводит все столбцы (включая True/False) в числа.
- Строки 94–95: сообщение в лог и результат.

```
 97      def _read_parameter(self, sheet: str) -> pd.DataFrame:
 98          table = pd.read_excel(self.path, sheet_name=sheet, header=1).iloc[1:]
 99          dates = {}
100          for column in table.columns:
101              if isinstance(column, (pd.Timestamp, date)):
102                  dates[column] = pd.Timestamp(column)
103              elif isinstance(column, str) and _DATE_HEADER.fullmatch(column.strip()):
104                  dates[column] = pd.to_datetime(column.strip(), format="%d.%m.%Y")
105          values = table[list(dates)].apply(pd.to_numeric, errors="coerce")
106          values.columns = pd.DatetimeIndex(list(dates.values()))
107          values.index = pd.MultiIndex.from_arrays(
108              [table["№ скв."].map(normalize_id), table["№ куста"].map(normalize_id)]
109          )
110          return values[~values.index.duplicated()]
```

`_read_parameter(sheet)` — один лист параметра → таблица «(номер, куст) × даты».

- Строка 98: читаем лист; `header=1` — заголовки во **второй** строке листа; `.iloc[1:]` — пропускаем ещё одну пустую строку под заголовком.
- Строки 99–104: находим столбцы-даты. В шапке они бывают двух видов: настоящая дата Excel (тогда pandas даёт `Timestamp`) или текст «01.01.2026». Оба варианта приводим к `Timestamp`; остальные столбцы (НГДУ, цех, номер…) пропускаются.

- Строка 105: берём только столбцы-даты и превращаем значения в числа; `errors="coerce"` — всё, что не число (текст, прочерки), становится NaN, а не ошибкой.
- Строка 106: названия столбцов заменяем на настоящие даты.
- Строки 107–109: строки таблицы индексируем парой (номер, куст), нормализованными через `normalize_id`. `MultiIndex` — индекс из двух уровней.
- Строка 110: если одна и та же скважина встречается дважды (дубли строк в Excel), оставляем первую. `~` — логическое НЕ.

```
112     def _read_states(self) -> pd.DataFrame:
113         sheet = pd.read_excel(self.path, sheet_name=self.STATE_SHEET, header=None)
114         is_state = sheet[4] == "Состояние"
115         info = sheet.shift(-1)[is_state][1].astype(str)  # строка «Прим»: «..., куст 2079, ...»
116         clusters = info.str.extract(r"куст\s+([^,]+)", expand=False).map(normalize_id)
117         values = sheet[is_state].iloc[:, 7:].apply(lambda column: column.astype(str).str.strip())
118         values.columns = pd.to_datetime(sheet.iloc[1, 7:].values)
119         values.index = pd.MultiIndex.from_arrays([sheet[is_state][3].map(normalize_id), clusters])
120         return values[~values.index.duplicated()]
```

`_read_states()` — лист «Прим». Он устроен иначе: на каждую скважину две строки — «Состояние» и «Прим» (примечание).

- Строка 113: читаем без заголовков (`header=None`) — столбцы нумеруются 0, 1, 2…
- Строка 114: `is_state` — строки, где в столбце 4 написано «Состояние».
- Строка 115: куст указан не в строке «Состояние», а в строке под ней, в тексте вида «КСП-11, куст 2079, ЗУ …». `shift(-1)` сдвигает таблицу на строку вверх, и для каждой строки «Состояние» мы видим текст следующей строки.
- Строка 116: регулярным выражением достаём номер куста после слова «куст» до запятой и нормализуем.
- Строка 117: значения состояний (с 8-го столбца, индекс 7) — в строки без пробелов.
- Строка 118: даты берутся из второй строки листа (индекс 1).
- Строки 119–120: индекс — (номер из столбца 3, куст), без дублей.

```
123 class LabelsReader:
124     """Лист «Результат__*»: мероприятия «Увеличение F», успешные, с обеими датами мониторинга."""
125
126     # Номер столбца (с нуля) -> имя. Шапка листа — 4 строки.
127     COLUMNS = {3: "cluster", 4: "number", 26: "event", 35: "date_done", 39: "success", 41: "start", 42:
"end"}
128
129     def __init__(self, path: Path, sheet: str | None = None) -> None:
130         self.path = Path(path)
131         self.sheet = sheet
```

Клас`LabelsReader` — разметка из листа «Результат__*».

`COLUMNS` — какие столбцы листа нужны (нумерация с нуля): 3 — куст, 4 — номер скважины, 26 — наименование мероприятия, 35 — дата выполнения, 39 — «успешно», 41 — дата начала мониторинга, 42 — дата окончания мониторинга. Шапка листа занимает 4 строки.

Конструктор: путь к файлу и необязательное имя листа (если не указано — найдётся автоматически).

```
133     def read(self) -> dict[tuple[str, str], list[Interval]]:
134         sheet = self.sheet or next(
135             name for name in pd.ExcelFile(self.path).sheet_names if name.startswith("Результат")
136         )
137         table = pd.read_excel(self.path, sheet_name=sheet, header=None, skiprows=4)
138         table = table[list(self.COLUMNS)].rename(columns=self.COLUMNS)
139
140         labels: dict[tuple[str, str], list[Interval]] = {}
141         for row in table.itertuples(index=False):
142             if str(row.event).strip().lower() != "увеличение f" or str(row.success).strip().lower() != "да":
143                 continue
144             start, end = parse_date(row.start), parse_date(row.end)
145             if start is None or end is None or end < start:
146                 continue
147             key = (normalize_id(row.number), normalize_id(row.cluster))
148             labels.setdefault(key, []).append(Interval(start, end, parse_date(row.date_done)))
149         logger.info("Разметка: %d мероприятий", sum(map(len, labels.values())))
150         return labels
```

`read()` — мероприятия по скважинам. Возвращает словарь «(номер, куст) → список `Interval`».

- Строки 134–136: если лист не указан, берём первый лист, имя которого начинается с «Результат». `next(...)` возвращает первый подходящий элемент генератора.
- Строки 137–138: читаем лист, пропустив 4 строки шапки; оставляем нужные столбцы и даём им понятные имена.
- Строки 141–143: идём по строкам. Пропускаем всё, кроме мероприятий «Увеличение F» со значением «да» в «успешно». `strip().lower()` — сравнение без учёта пробелов и регистра.
- Строки 144–146: разбираем даты начала и окончания; если какой-то нет или окончание раньше начала — пропускаем (такие строки бесполезны для обучения).
- Строки 147–148: добавляем мероприятие в список своей скважины. `setdefault(key, [])` — взять список по ключу, а если ключа нет, создать пустой.
- Строка 149: в лог — сколько мероприятий набралось (на текущих данных 117).

```
153 class WellRepository:
154     """Скважины с привязанной разметкой; результат чтения Excel кэшируется на диске."""
155 
156     def __init__(self, measurements: Path, results: Path | None, cache_dir: Path | None = None) -> None:
157         self.measurements = Path(measurements)
158         self.results = Path(results) if results else None
159         self.cache_dir = cache_dir or Path(tempfile.gettempdir()) / "gtm_ml_freq_cache"
```

Класс `WellRepository` — «склад» скважин: читает замеры, привязывает к ним разметку и кэширует результат.

- `measurements` — файл замеров;
- `results` — файл разметки или `None` (для поиска мероприятий без разметки);
- `cache_dir` — папка кэша; по умолчанию во временной папке системы: `%TEMP%/gtm_ml_freq_cache`.

```
161     def load(self) -> list[WellSeries]:
162         cache = self.cache_dir / f"wells_{self._fingerprint()}.pkl"
163         if cache.exists():
164             logger.info("Данные из кэша: %s", cache)
165             with cache.open("rb") as file:
166                 return pickle.load(file)
167 
168         wells = MeasurementsReader(self.measurements).read()
169         if self.results:
170             self._attach(wells, LabelsReader(self.results).read())
171         self.cache_dir.mkdir(parents=True, exist_ok=True)
172         with cache.open("wb") as file:
173             pickle.dump(wells, file)
174         return wells
```

`load()` — получить скважины.

- Строка 162: имя файла кэша содержит «отпечаток» исходных файлов (см. `_fingerprint`).
- Строки 163–166: если такой кэш уже есть — просто загружаем его. Это доли секунды вместо ~20 секунд чтения Excel.
- Строки 168–170: иначе читаем Excel и, если задан файл разметки, привязываем разметку к скважинам.
- Строки 171–173: сохраняем результат в кэш. `with ... open("wb")` — открыть файл на двоичную запись и гарантированно закрыть после.

Важно: `pickle`-кэш создаётся самой программой из ваших файлов. Загружать чужие pickle-файлы небезопасно — они могут выполнить код.

```
176     @staticmethod
177     def _attach(wells: Iterable[WellSeries], labels: dict[tuple[str, str], list[Interval]]) -> None:
178         by_key = {(well.number, well.cluster): well for well in wells}
179         by_number: dict[str, list[WellSeries]] = {}
180         for well in by_key.values():
181             by_number.setdefault(well.number, []).append(well)
182         lost = 0
183         for (number, cluster), intervals in labels.items():
184             well = by_key.get((number, cluster))
185             if well is None and len(by_number.get(number, [])) == 1:
186                 well = by_number[number][0]  # куст записан иначе, но номер уникален
187             if well is None:
188                 lost += len(intervals)
189                 continue
190             well.labels.extend(intervals)
191         if lost:
192             logger.warning("Не найдено скважин для %d мероприятий разметки", lost)
```

`_attach` — привязать разметку к скважинам. `@staticmethod` — методу не нужен `self`.

- Строка 178: словарь «(номер, куст) → скважина» для быстрого поиска.
- Строки 179–181: второй словарь «номер → список скважин с этим номером».
- Строки 183–186: для каждой группы мероприятий ищем скважину по паре (номер, куст). Если не нашли, но номер на месторождении **уникален** — берём скважину по одному номеру: значит, куст в разметке записан иначе, чем в замерах.
- Строки 187–189: скважина так и не нашлась — считаем потерянные мероприятия.
- Строка 190: добавляем мероприятия в `labels` скважины.
- Строки 191–192: предупреждение в лог, если что-то потерялось. На текущих данных потерь нет.

```
194     def _fingerprint(self) -> str:
195         parts = []
196         for path in (self.measurements, self.results):
197             if path is not None:
198                 stat = path.stat()
199                 parts.append(f"{path.resolve()}:{stat.st_size}:{stat.st_mtime_ns}")
200         return hashlib.sha1("|".join(parts).encode()).hexdigest()[:12]
```

`_fingerprint` — «отпечаток» файлов для имени кэша. Для каждого файла собирается строка «полный путь : размер : время изменения», всё склеивается и хешируется SHA-1, берутся первые 12 символов.

Смысл: стоит изменить или заменить Excel-файл — меняется размер или время, меняется отпечаток, кэш автоматически считается устаревшим и данные перечитываются. Удалять кэш вручную не нужно.

Файл gtm_detector/ml_freq/data/__init__.py

```
1  """Данные: чтение Excel и модели скважин/мероприятий."""
2
3  from .models import SERIES_COLUMNS, Interval, WellSeries
4  from .readers import LabelsReader, MeasurementsReader, WellRepository, normalize_id, parse_date
5
6  __all__ = [
7      "SERIES_COLUMNS",
8      "Interval",
9      "LabelsReader",
10     "MeasurementsReader",
11     "WellRepository",
12     "WellSeries",
13     "normalize_id",
14     "parse_date",
15 ]
```

`data/__init__.py` собирает публичные имена подпакета в одно место: другие модули пишут `from ..data import WellSeries`, а не `from ..data.models import WellSeries`. Внутреннее устройство подпакета можно менять, не трогая импорты снаружи.

`__all__` — явный список публичных имён: что импортируется при `from ... import *` и что читатель должен считать «внешним интерфейсом».

features/builder.py — признаки по дням

Файл gtm_detector/ml_freq/features/builder.py

Превращает сырые ряды скважины в числа, удобные моделям. Главные принципы: всё в относительных величинах (чтобы скважины разного масштаба были сопоставимы), без пропусков, в диапазоне от −3 до 3.

```
"""Признаки по дням для моделей.

* :meth:`FeatureBuilder.sequence` — матрица «день × канал» для Transformer:
  модель сама смотрит на соседние дни.
* :meth:`FeatureBuilder.tabular` — те же каналы плюс их сдвиги на ±1…14 дней
  и скользящие средние: классическим моделям каждый день виден отдельно,
  контекст нужно подать явно.
"""
```

Описание двух видов признаков:

- `sequence` — матрица «день × канал» для Transformer: модель сама смотрит на соседние дни через внимание;
- `tabular` — для моделей scikit-learn: они видят каждый день отдельно, поэтому контекст (значения соседних дней) нужно добавить явно — сдвигами и скользящими средними.

```
from __future__ import annotations

import numpy as np
import pandas as pd

from ..data import WellSeries

CLIP = 3.0
```

Импорты и константа `CLIP = 3.0`: все признаки в конце обрезаются до диапазона [−3, 3]. Одна ошибка ввода (например, дебит в 100 раз больше обычного) не создаст гигантского числа, которое сбило бы обучение.

```
class FeatureBuilder:
    """Строит признаки одной скважины. Без состояния — один экземпляр на все скважины."""

    MIN_WORKING_FREQ = 10.0  # Гц: ниже насос считается остановленным
    MIN_STEP = 0.1  # Гц: подъём частоты, после которого считаются признаки «от скачка»
    WINDOW = 30  # дней: «недавний» уровень параметров
```

Класс `FeatureBuilder`. «Без состояния» — он ничего не запоминает между вызовами, поэтому один объект годится для всех скважин.

- `MIN_WORKING_FREQ = 10.0` — частота ниже 10 Гц означает, что насос фактически стоит;
- `MIN_STEP = 0.1` — подъём частоты от 0.1 Гц считается «скачком», от которого отсчитываются признаки «после скачка»;
- `WINDOW = 30` — «недавнее прошлое» для сравнения: 30 дней.

```
    SEQUENCE_CHANNELS = (
        "working",
        "f_rel",
        "f_diff",
        "f_above_min30",
        "f_below_max30",
        "liq_vs_30d",
        "oil_vs_30d",
        "wc_vs_30d",
        "pzab_vs_30d",
        "days_since_step",
        "step_size",
        "liq_vs_step",
        "oil_vs_step",
        "wc_vs_step",
        "liq_missing",
    )
```

`SEQUENCE_CHANNELS` — 15 каналов (признаков) каждого дня. Порядок важен: в этом порядке они идут в матрицу и в сеть. Смысл каждого объяснён в `_channels` ниже.

```
44      SHIFTED_CHANNELS = (
45          "working",
46          "f_diff",
47          "f_above_min30",
48          "liq_vs_30d",
49          "oil_vs_30d",
50          "wc_vs_30d",
51          "liq_vs_step",
52          "oil_vs_step",
53          "wc_vs_step",
54      )
55      OFFSETS = (-14, -7, -3, -1, 1, 3, 7, 14)
```

SHIFTED_CHANNELS — 9 каналов, для которых модели scikit-learn получают ещё и значения соседних дней. Выбраны самые информативные: работа, изменение частоты, отклонения дебитов и обводнённости.

OFFSETS — на сколько дней сдвигать: 14, 7, 3, 1 день назад (отрицательные) и вперёд (положительные). Дальние сдвиги дают «общую картину», ближние — точную границу.

```
57      @property
58      def sequence_size(self) -> int:
59          return len(self.SEQUENCE_CHANNELS)
60
61      @property
62      def tabular_names(self) -> list[str]:
63          names = list(self.SEQUENCE_CHANNELS)
64          names += [f"{c}@{o:+d}" for c in self.SHIFTED_CHANNELS for o in self.OFFSETS]
65          names += [f"{c}_mean_{side}7" for c in self.SHIFTED_CHANNELS for side in ("past", "next")]
66          names += ["f_diff_max_pm3"]
67          return names
```

- `sequence_size` — число каналов (15); по нему Transformer узнаёт размер входа.
- `tabular_names` — имена всех 106 табличных признаков в том же порядке, что и столбцы матрицы. 15 базовых + 9 × 8 сдвигов (например `oil_vs_step@+7` — значение через 7 дней; формат `{o:+d}` всегда ставит знак) + 9 × 2 средних (за 7 дней до и после) + 1 (`f_diff_max_pm3`) = 106. Нужны, чтобы понимать, что значит каждый столбец, например при анализе важности признаков.

```
69      def sequence(self, well: WellSeries) -> np.ndarray:
70          channels = self._channels(well)
71          return channels[list(self.SEQUENCE_CHANNELS)].to_numpy(dtype=np.float32)
```

sequence(well) — считает каналы и возвращает матрицу «дни × 15» типа `float32` (тип, который ждёт PyTorch). `channels[list(...)]` гарантирует нужный порядок столбцов.

```
73      def tabular(self, well: WellSeries) -> np.ndarray:
74          channels = self._channels(well)
75          parts = [channels[list(self.SEQUENCE_CHANNELS)]]
76          shifted = channels[list(self.SHIFTED_CHANNELS)]
77          for offset in self.OFFSETS:
78              # offset > 0 — значение через offset дней (детекция офлайн, будущее известно).
79              parts.append(shifted.shift(-offset).fillna(0.0).add_suffix(f"@{offset:+d}"))
80          parts.append(shifted.rolling(7, min_periods=1).mean().add_suffix("_mean_past7"))
81          parts.append(shifted[::-1].rolling(7, min_periods=1).mean()[::-1].add_suffix("_mean_next7"))
82          parts.append(
83              channels["f_diff"]
84              .rolling(7, center=True, min_periods=1)
85              .max()
86              .rename("f_diff_max_pm3")
87              .to_frame()
88          )
89          table = pd.concat(parts, axis=1)
90          return table.to_numpy(dtype=np.float32)
```

tabular(well) — табличные признаки для scikit-learn.

- Строки 74–75: базовые 15 каналов.
- Строка 76: из них 9 каналов для сдвигов.
- Строки 77–79: для каждого сдвига `shift(-offset)` смещает ряд. При `offset = +7` в строке дня T окажется значение дня T+7 — модель «видит будущее». Это допустимо: поиск офлайн, весь ряд известен. На краях ряда пустые значения → 0. `add_suffix` дописывает к именам столбцов `@+7`.
- Строка 80: скользящее среднее за последние 7 дней (включая текущий) — «что было недавно».
- Строка 81: среднее за **следующие** 7 дней. Трюк: развернуть ряд (`[::-1]`), посчитать обычное скользящее среднее, развернуть обратно.

- Строки 82–88: максимум изменения частоты в окне ±3 дня вокруг дня (`center=True`) — «был ли рядом скачок частоты».
- Строки 89–90: склеиваем все части по столбцам и возвращаем матрицу `float32`.

```
92      def _channels(self, well: WellSeries) -> pd.DataFrame:
93          frame = well.frame
94          working = (frame["state"] > 0) & (frame["frequency"] > self.MIN_WORKING_FREQ)
95          frequency = frame["frequency"].where(working).ffill().bfill()
96          reference = frequency[working].median() if working.any() else np.nan
97          reference = reference if reference and not np.isnan(reference) else 1.0
98          f_rel = (frequency / reference - 1).fillna(0.0)
99
100         out = pd.DataFrame(index=frame.index)
101         out["working"] = working.astype(float)
102         out["f_rel"] = f_rel
103         out["f_diff"] = f_rel.diff().fillna(0.0) * 10  # 1 Гц при 50 Гц -> 0.2
104         out["f_above_min30"] = (f_rel - f_rel.rolling(self.WINDOW, min_periods=1).min()) * 10
105         out["f_below_max30"] = (f_rel - f_rel.rolling(self.WINDOW, min_periods=1).max()) * 10
106         out["liq_vs_30d"] = self._relative_to_past(frame["debit_liquid"], working)
107         out["oil_vs_30d"] = self._relative_to_past(frame["debit_oil"], working)
108         out["wc_vs_30d"] = self._difference_to_past(frame["watercut"], working) / 10
109         out["pzab_vs_30d"] = self._relative_to_past(frame["pressure_zab"], working)
110         out = out.join(self._since_last_step(frame, frequency, working))
111         out["liq_missing"] = frame["debit_liquid"].isna().astype(float)
112         return out.fillna(0.0).clip(-CLIP, CLIP)
```

`_channels(well)` — расчёт 15 базовых каналов. Сердце модуля.

- Строка 94: `working` — скважина работает: состояние «работает» **и** частота выше 10 Гц.
- Строка 95: частота только в рабочие дни; в остальные дни подставляется последнее известное значение (`ffill`), в самом начале — первое следующее (`bfill`). Так простой не выглядит как «падение частоты до нуля».
- Строки 96–97: `reference` — медианная частота скважины в рабочие дни. Если данных нет — 1.0, чтобы не делить на ноль/NaN.
- Строка 98: `f_rel` — частота относительно медианы, минус 1. Для двигателя на 50 Гц подъём на 1 Гц даёт +0.02, для двигателя на 150 Гц подъём на 3 Гц — тоже +0.02. Скважины разного типа становятся сопоставимы.
- Строка 101: канал `working` (1/0).
- Строка 103: `f_diff` — изменение `f_rel` за сутки, ×10, чтобы числа были порядка 0.1–1, а не 0.01. День подъёма частоты даёт заметный положительный всплеск.
- Строки 104–105: на сколько частота выше минимума и ниже максимума за последние 30 дней. Первое — «частоту недавно подняли», второе — «недавно опускали».
- Строки 106–107: Qж и Qн относительно медианы предыдущих 30 рабочих дней: +0.1 — дебит на 10% выше обычного.
- Строка 108: обводнённость — разность с её средним за 30 дней (в процентных пунктах), /10.
- Строка 109: забойное давление относительно прошлого: при подъёме частоты оно обычно падает.
- Строка 110: присоединяем 5 признаков «после последнего скачка частоты» (см. ниже).
- Строка 111: `liq_missing` — замера Qж в этот день нет (1/0): модели полезно знать, что нули на самом деле пропуски.
- Строка 112: пропуски → 0, всё обрезается в [−3, 3].

```
114     def _relative_to_past(self, series: pd.Series, working: pd.Series) -> pd.Series:
115         past = series.where(working).shift(1).rolling(self.WINDOW, min_periods=3).median()
116         return (series / past.replace(0, np.nan) - 1).where(working)
117
118     def _difference_to_past(self, series: pd.Series, working: pd.Series) -> pd.Series:
119         past = series.where(working).shift(1).rolling(self.WINDOW, min_periods=3).mean()
120         return (series - past).where(working)
```

Два помощника для сравнения с прошлым:

- `_relative_to_past` — отношение к прошлому минус 1. `where(working)` убирает дни простоя; `shift(1)` — берём только **предыдущие** дни, без текущего (иначе день сравнивался бы сам с собой);

`rolling(30, min_periods=3).median()` — медиана за 30 дней, если в окне есть хотя бы 3 значения. `replace(0, nan)` — защита от деления на ноль. Результат тоже только для рабочих дней;

- `_difference_to_past` — то же, но разность со средним. Подходит для обводнённости: она в процентах, и «на сколько пунктов выросла» понятнее, чем «во сколько раз».

```
122     def _since_last_step(self, frame: pd.DataFrame, frequency: pd.Series, working: pd.Series) ->
pd.DataFrame:
123         """Признаки относительно последнего скачка частоты вверх (как у детектора на правилах)."""
124         smoothed = frequency.rolling(3, center=True, min_periods=1).median()
125         is_step = (smoothed.diff() >= self.MIN_STEP) & working
126         positions = np.flatnonzero(is_step.to_numpy())
127 
128         n = len(frame)
129         result = {
130             name: np.zeros(n)
131             for name in ("days_since_step", "step_size", "liq_vs_step", "oil_vs_step", "wc_vs_step")
132         }
133         result["days_since_step"][:] = 1.0  # «давно не было скачка»
134         liquid, oil = frame["debit_liquid"].to_numpy(), frame["debit_oil"].to_numpy()
135         watercut, values = frame["watercut"].to_numpy(), smoothed.to_numpy()
136         boundaries = list(positions) + [n]
137         for step, next_step in zip(boundaries[:-1], boundaries[1:], strict=True):
138             before = max(step - 1, 0)
139             span = slice(step, next_step)
140             days = np.arange(next_step - step)
141             result["days_since_step"][span] = np.minimum(days, 120) / 120
142             result["step_size"][span] = (values[step] - values[before]) / max(values[before], 1.0) * 10
143             result["liq_vs_step"][span] = _ratio(liquid[span], liquid[before])
144             result["oil_vs_step"][span] = _ratio(oil[span], oil[before])
145             result["wc_vs_step"][span] = (watercut[span] - watercut[before]) / 10
146         return pd.DataFrame(result, index=frame.index)
```

`_since_last_step` — признаки относительно последнего подъёма частоты. Это ровно та информация, на которую опирается детектор на правилах: «что стало с дебитом по сравнению с днём перед подъёмом частоты».

- Строка 124: частота сглаживается медианой за 3 дня, чтобы одиночный выброс не считался скачком.
- Строки 125–126: дни скачка — сглаженная частота выросла на 0.1 Гц и больше в рабочий день. `positions` — номера этих дней.
- Строки 128–133: заготовки пяти признаков (нули). `days_since_step` по умолчанию 1.0 — «скачка давно не было» (максимум шкалы).
- Строки 134–135: ряды как массивы NumPy — так быстрее.
- Строка 136: границы отрезков: каждый скачок до следующего скачка (последний — до конца ряда).
- Строки 137–145: для каждого отрезка «от скачка до следующего скачка»:
  - `before` — день перед скачком (базовый режим «до»);
  - `days_since_step` — сколько дней прошло со скачка, не больше 120, делённое на 120 (шкала 0…1);
  - `step_size` — величина скачка относительно частоты «до», ×10;
  - `liq_vs_step`, `oil_vs_step` — дебит жидкости и нефти относительно дня перед скачком;
  - `wc_vs_step` — изменение обводнённости относительно дня перед скачком, /10.
- Строка 146: результат — таблица на тех же датах.

Пример: частоту подняли 26.03, Qж до этого был 320. Через неделю Qж = 339 → `liq_vs_step` = 339/320 − 1 ≈ +0.06, `days_since_step` = 7/120.

```
149 def _ratio(values: np.ndarray, base: float) -> np.ndarray:
150     if base is None or np.isnan(base) or base == 0:
151         return np.zeros_like(values)
152     return values / base - 1
```

`_ratio(values, base)` — «значения относительно базы, минус 1». Если базы нет (NaN, ноль) — нули, чтобы не получить бесконечности. Функция модуля, а не метод: ей не нужен объект.

Файл gtm_detector/ml_freq/features/__init__.py

```
"""Признаки по дням для моделей."""

from .builder import FeatureBuilder

__all__ = ["FeatureBuilder"]
```

Делает `FeatureBuilder` доступным как `from ..features import FeatureBuilder`. `__all__` — список публичных имён.

evaluation/decoding.py — вероятности → мероприятия

Файл gtm_detector/ml_freq/evaluation/decoding.py

Модели выдают для каждого дня вероятность эффекта. Декодер превращает этот ряд чисел в список мероприятий с датами. Он общий для всех моделей — это делает сравнение честным: модели различаются только качеством вероятностей.

```
"""Вероятности по дням -> мероприятия с датами начала и окончания."""

from __future__ import annotations

from collections.abc import Sequence
from datetime import date

import numpy as np
import pandas as pd

from ..config import DecoderConfig
from ..data import Interval
```

Импорты: `DecoderConfig` — настройки (порог, сглаживание, склейка), `Interval` — результат.

```
class SegmentDecoder:
    """Общий для всех моделей шаг: одинаковые правила — честное сравнение.

    1. Вероятности сглаживаются скользящим средним (``smooth_days``).
    2. Дни выше ``threshold`` — «в эффекте».
    3. Разрывы не длиннее ``max_gap`` дней склеиваются.
    4. Периоды короче ``min_length`` дней отбрасываются.
    5. Период, дошедший до конца данных, — эффект продолжается (``end=None``).
    """

    def __init__(self, config: DecoderConfig | None = None) -> None:
        self.config = config or DecoderConfig()
```

Класс `SegmentDecoder`. Docstring перечисляет 5 шагов алгоритма. Конструктор берёт настройки; если не переданы — значения по умолчанию из `config.py`.

```
    def decode(self, proba: np.ndarray, dates: Sequence[date]) -> list[Interval]:
        cfg = self.config
        smoothed = pd.Series(proba).rolling(cfg.smooth_days, center=True, min_periods=1).mean().to_numpy()
        active = smoothed > cfg.threshold
        runs = self._runs(active)
        runs = self._merge(runs, cfg.max_gap)
        last = len(proba) - 1
        return [
            Interval(start=dates[begin], end=None if end == last else dates[end])
            for begin, end in runs
            if end - begin + 1 >= cfg.min_length
        ]
```

`decode(proba, dates)` — главный метод.

- Строка 30: **сглаживание**. Ряд вероятностей в pandas, скользящее среднее за 5 дней с центром на текущем дне (`center=True`: 2 дня до, день, 2 дня после). `min_periods=1` — на краях ряда считать по тому, что есть. Убирает одиночные всплески и провалы.
- Строка 31: **порог** — массив True/False: день «в эффекте», если сглаженная вероятность больше 0.5.
- Строка 32: **отрезки** — непрерывные куски True (см. `_runs`).
- Строка 33: **склейка** отрезков с маленькими разрывами (см. `_merge`).
- Строка 34: номер последнего дня ряда.

- Строки 35–39: каждый отрезок длиной не меньше `min_length` превращается в `Interval`: начало — дата первого дня, окончание — дата последнего. Если отрезок доходит до последнего дня данных, окончание `None` — «эффект продолжается»: мы не знаем, когда он закончится.

```
41      @staticmethod
42      def _runs(active: np.ndarray) -> list[tuple[int, int]]:
43          edges = np.diff(np.r_[0, active.astype(int), 0])
44          starts, stops = np.flatnonzero(edges == 1), np.flatnonzero(edges == -1) - 1
45          return list(zip(starts.tolist(), stops.tolist(), strict=True))
```

`_runs(active)` — найти непрерывные отрезки True. Векторный трюк без цикла по дням:

- `active.astype(int)` — True/False в 1/0;
- `np.r_[0, ..., 0]` — дописать по нулю в начало и конец, чтобы отрезки у краёв тоже имели «вход» и «выход»;
- `np.diff` — разности соседних элементов: +1 там, где начинается отрезок (0→1), −1 — сразу после его конца (1→0);
- `starts` — позиции +1 (начала), `stops` — позиции −1 минус 1 (последний день отрезка);
- результат — список пар (начало, конец).

Пример: `[0,1,1,0,1]` → с нулями `[0,0,1,1,0,1,0]` → разности `[0,1,0,-1,1,-1]` → отрезки (1,2) и (4,4).

```
47      @staticmethod
48      def _merge(runs: list[tuple[int, int]], max_gap: int) -> list[tuple[int, int]]:
49          merged: list[tuple[int, int]] = []
50          for begin, end in runs:
51              if merged and begin - merged[-1][1] - 1 <= max_gap:
52                  merged[-1] = (merged[-1][0], end)
53              else:
54                  merged.append((begin, end))
55          return merged
```

`_merge(runs, max_gap)` — склейка близких отрезков. Идём по отрезкам по порядку. Если разрыв между концом предыдущего склеенного отрезка и началом текущего (`begin − конец − 1` — число дней между ними) не больше `max_gap`, расширяем предыдущий до конца текущего. Иначе текущий становится новым отрезком.

Пример при `max_gap = 3`: (5,14) и (16,19) — разрыв 1 день → (5,19).

evaluation/metrics.py — сравнение с разметкой и метрики

Файл gtm_detector/ml_freq/evaluation/metrics.py

Два класса: `EventMatcher` решает, какое найденное мероприятие соответствует какому размеченному, а `Score` копит результаты и считает метрики.

```
1   """Сопоставление найденных мероприятий с разметкой и метрики."""
2
3   from __future__ import annotations
4
5   from collections.abc import Sequence
6   from dataclasses import dataclass, field
7
8   import numpy as np
9
10  from ..data import Interval
11
12
13  def _share(errors: Sequence[int], days: int) -> float:
14      return float(np.mean(np.abs(errors) <= days)) if len(errors) else float("nan")
15
16
17  def _mae(errors: Sequence[int]) -> float:
18      return float(np.mean(np.abs(errors))) if len(errors) else float("nan")
```

Две вспомогательные функции:

- `_share(errors, days)` — доля ошибок, по модулю не больше `days` дней. Например, при `days = 0` — доля точных попаданий. `np.abs(errors) <= days` даёт True/False, среднее от них — доля True. Пустой список — NaN («не определено»), а не ошибка деления на ноль;

- `_mae(errors)` — средняя абсолютная ошибка в днях (Mean Absolute Error).

```
21 @dataclass
22 class Score:
23     """Метрики уровня мероприятий; складываются оператором ``+``."""
24 
25     true_positives: int = 0
26     false_positives: int = 0
27     false_negatives: int = 0
28     start_errors: list[int] = field(default_factory=list)  # найдено – разметка, дней
29     end_errors: list[int] = field(default_factory=list)
30     open_ends: int = 0  # в разметке конец есть, модель считает эффект продолжающимся
```

Dataclass `Score` — копилка результатов.

- `true_positives` (TP) — размеченное мероприятие найдено;
- `false_positives` (FP) — найдено то, чего в разметке нет;
- `false_negatives` (FN) — размеченное мероприятие не найдено;
- `start_errors` — для каждого найденного: «дата начала у модели − дата начала в разметке» в днях. Отрицательное — модель поставила раньше, положительное — позже;
- `end_errors` — то же для окончания;
- `open_ends` — в разметке окончание есть, а модель считает, что эффект ещё продолжается.

Списки — через `field(default_factory=list)`, чтобы у каждого объекта был свой.

```
32     @property
33     def recall(self) -> float:
34         total = self.true_positives + self.false_negatives
35         return self.true_positives / total if total else 0.0
36 
37     @property
38     def precision(self) -> float:
39         found = self.true_positives + self.false_positives
40         return self.true_positives / found if found else 0.0
41 
42     @property
43     def f1(self) -> float:
44         p, r = self.precision, self.recall
45         return 2 * p * r / (p + r) if p + r else 0.0
```

Метрики-свойства:

- **`recall` (полнота)** = TP / (TP + FN) — какую долю размеченных мероприятий модель нашла. Главная метрика здесь: разметка надёжна в том, что в ней есть;
- **`precision` (точность)** = TP / (TP + FP) — какая доля найденного есть в разметке. Здесь занижена: в разметке не все реальные мероприятия, поэтому FP — не обязательно ошибки;
- **`f1`** = 2·P·R / (P + R) — гармоническое среднее полноты и точности, одно число для сравнения.

Везде защита от деления на ноль: если знаменатель 0 — результат 0.

```
47     def __add__(self, other: Score) -> Score:
48         return Score(
49             self.true_positives + other.true_positives,
50             self.false_positives + other.false_positives,
51             self.false_negatives + other.false_negatives,
52             self.start_errors + other.start_errors,
53             self.end_errors + other.end_errors,
54             self.open_ends + other.open_ends,
55         )
```

`__add__` — сложение оценок оператором `+`. Счётчики складываются, списки ошибок склеиваются. Это позволяет считать `Score` по каждой скважине отдельно и просто суммировать: `total = total + score_скважины` (или `total += ...`).

```
57      def as_dict(self) -> dict[str, float]:
58          return {
59              "найдено размеченных (recall)": self.recall,
60              "точность (precision)": self.precision,
61              "F1": self.f1,
62              "начало: точно": _share(self.start_errors, 0),
63              "начало: ±1 дн": _share(self.start_errors, 1),
64              "начало: ±3 дн": _share(self.start_errors, 3),
65              "начало: MAE, дн": _mae(self.start_errors),
66              "конец: точно": _share(self.end_errors, 0),
67              "конец: ±3 дн": _share(self.end_errors, 3),
68              "конец: ±7 дн": _share(self.end_errors, 7),
69              "конец: MAE, дн": _mae(self.end_errors),
70              "TP": self.true_positives,
71              "FN": self.false_negatives,
72              "FP": self.false_positives,
73              "не закрыт конец": self.open_ends,
74          }
```

`as_dict()` — все метрики словарём с русскими названиями: строка итоговой таблицы сравнения в Excel. Включает полноту, точность, F1, доли точных/±1/±3 по началу, точных/±3/±7 по окончанию, средние ошибки в днях и сырые счётчики TP/FN/FP.

```
77  class EventMatcher:
78      """Один-к-одному по дате начала (|Δ| ≤ ``start_tolerance``), сначала ближайшие пары."""
79
80      def __init__(self, start_tolerance: int = 7) -> None:
81          self.start_tolerance = start_tolerance
```

Класс `EventMatcher`. Пара «найдено — размечено» засчитывается, если даты начала отличаются не больше чем на `start_tolerance` (7) дней.

```
83      def match(
84          self, predicted: Sequence[Interval], labels: Sequence[Interval]
85      ) -> list[tuple[Interval, Interval | None]]:
86          candidates = sorted(
87              (abs((p.start - t.start).days), i, j)
88              for i, p in enumerate(predicted)
89              for j, t in enumerate(labels)
90              if abs((p.start - t.start).days) <= self.start_tolerance
91          )
92          used_predicted: set[int] = set()
93          found: dict[int, int] = {}
94          for _, i, j in candidates:
95              if i not in used_predicted and j not in found:
96                  used_predicted.add(i)
97                  found[j] = i
98          return [(label, predicted[found[j]] if j in found else None) for j, label in enumerate(labels)]
```

`match(predicted, labels)` — сопоставление один-к-одному.

- Строки 86–91: все возможные пары (найденное i, размеченное j) с разницей начал не больше 7 дней, в виде кортежей (разница, i, j). `sorted` упорядочивает по разнице — сначала самые близкие пары.
- Строки 92–97: жадный проход: берём пару, если ни найденное i, ни размеченное j ещё не заняты. Так одно найденное не может закрыть два размеченных и наоборот, а при спорах побеждает более близкая пара.
- Строка 98: для каждой метки разметки — найденное мероприятие или `None`, если пары нет.

Пример: разметка начинается 10-го; модель нашла 11-го и 12-го. Засчитается 11-е (разница 1), 12-е останется лишним (FP).

```
100     def score(self, predicted: Sequence[Interval], labels: Sequence[Interval]) -> Score:
101         pairs = self.match(predicted, labels)
102         score = Score()
103         for label, hit in pairs:
104             if hit is None:
105                 score.false_negatives += 1
106                 continue
107             score.true_positives += 1
108             score.start_errors.append((hit.start - label.start).days)
109             if label.end is not None:
110                 if hit.end is None:
111                     score.open_ends += 1
112                 else:
113                     score.end_errors.append((hit.end - label.end).days)
114         score.false_positives = len(predicted) - score.true_positives
115         return score
```

`score(predicted, labels)` — метрики по одной скважине.

- Для каждой метки: нет пары → FN. Есть пара → TP, плюс ошибка начала в днях.
- Ошибка окончания считается, только если окончание есть в разметке. Если у модели окончания нет (эффект «продолжается») — это не ошибка в днях, а отдельный счётчик `open_ends`.
- Строка 114: FP = всё найденное минус сопоставленное.

evaluation/experiment.py — перекрёстная проверка

Файл gtm_detector/ml_freq/evaluation/experiment.py

Честная оценка моделей: каждая модель проверяется только на скважинах, которых не видела при обучении. Разбиение скважин одинаково для всех моделей.

```
1  """Сравнение моделей: k-fold по скважинам, одинаковые разбиения для всех моделей."""
2
3  from __future__ import annotations
4
5  import logging
6  import time
7  from collections.abc import Callable, Mapping, Sequence
8  from dataclasses import dataclass, field
9
10 import numpy as np
11 from sklearn.metrics import roc_auc_score
12
13 from ..config import ExperimentConfig
14 from ..data import WellSeries
15 from ..models import EventSegmenter
16 from .decoding import SegmentDecoder
17 from .metrics import EventMatcher, Score
18
19 logger = logging.getLogger(__name__)
```

Импорты: `time` — замер длительности обучения; `Callable`, `Mapping`, `Sequence` — аннотации; `roc_auc_score` из scikit-learn — метрика качества вероятностей; свои классы конфигурации, данных, моделей, декодера и метрик.

```
22 @dataclass
23 class ModelResult:
24     """Итог перекрёстной проверки одной модели."""
25
26     name: str
27     title: str
28     score: Score = field(default_factory=Score)
29     day_auc: float = float("nan")  # ROC-AUC по дням: качество вероятностей до декодирования
30     train_seconds: float = 0.0
31     predictions: list[dict] = field(default_factory=list)  # строки «метка — найдено»
32
33     def as_row(self) -> dict:
34         return {
35             "модель": self.title,
36             **self.score.as_dict(),
37             "ROC-AUC по дням": self.day_auc,
38             "обучение, с (все фолды)": round(self.train_seconds, 1),
39         }
```

Dataclass `ModelResult` — итог проверки одной модели.

- `name`, `title` — имя и название модели;
- `score` — суммарные метрики по всем фолдам;
- `day_auc` — ROC-AUC по дням: насколько хорошо вероятности отделяют дни эффекта от остальных **до** декодирования. 1.0 — идеально, 0.5 — как монетка. Помогает понять, что плохо: сами вероятности или превращение их в даты;
- `train_seconds` — суммарное время обучения;
- `predictions` — строки «метка — найдено» для листа Excel.

`as_row()` — одна строка таблицы сравнения. `**self.score.as_dict()` «распаковывает» словарь метрик внутрь нового словаря.

```
42  class CrossValidator:
43      """k-fold по скважинам: модель не видит скважины, на которых её проверяют."""
44
45      def __init__(self, config: ExperimentConfig | None = None) -> None:
46          self.config = config or ExperimentConfig()
47          self.decoder = SegmentDecoder(self.config.decoder)
48          self.matcher = EventMatcher(self.config.start_tolerance)
```

Класс `CrossValidator`. Конструктор создаёт один декодер и один сопоставитель на всё время проверки — с настройками из `ExperimentConfig`.

```
50      def folds(self, wells: Sequence[WellSeries]) -> list[list[int]]:
51          order = np.random.default_rng(self.config.seed).permutation(len(wells))
52          return [part.tolist() for part in np.array_split(order, self.config.folds)]
```

`folds(wells)` — разбиение скважин на части. `permutation` — случайная перестановка номеров скважин (зерно `seed`: каждый запуск одинаковый), `array_split` делит её на 5 почти равных частей. Результат — 5 списков номеров скважин.

Разбиение по **скважинам**, а не по дням: иначе соседние дни одной скважины попали бы и в обучение, и в проверку, и модель «подглядывала» бы.

```
54      def run(
55          self, wells: Sequence[WellSeries], models: Mapping[str, Callable[[], EventSegmenter]]
56      ) -> list[ModelResult]:
57          folds = self.folds(wells)
58          return [self._evaluate(name, factory, wells, folds) for name, factory in models.items()]
```

`run(wells, models)` — разбиение считается один раз и используется для всех моделей; затем каждая модель оценивается отдельно. `models` — словарь «имя → фабрика модели» (функция, создающая новую необученную модель).

```
60      def _evaluate(
61          self,
62          name: str,
63          factory: Callable[[], EventSegmenter],
64          wells: Sequence[WellSeries],
65          folds: list[list[int]],
66      ) -> ModelResult:
67          result: ModelResult | None = None
68          targets, probas = [], []
69          for k, test_idx in enumerate(folds):
70              test_set = set(test_idx)
71              train = [well for i, well in enumerate(wells) if i not in test_set]
72              model = factory()
73              if result is None:
74                  result = ModelResult(name=name, title=model.title)
75              started = time.perf_counter()
76              model.fit(train)
77              result.train_seconds += time.perf_counter() - started
78
79              for i in test_idx:
80                  well = wells[i]
81                  proba = model.predict_proba(well)
82                  targets.append(well.target())
83                  probas.append(proba)
84                  predicted = self.decoder.decode(proba, well.dates)
85                  result.score += self.matcher.score(predicted, well.labels)
86                  result.predictions.extend(self._rows(well, predicted))
87              logger.info("%s: фолд %d/%d готов", name, k + 1, len(folds))
88
89          y, p = np.concatenate(targets), np.concatenate(probas)
90          result.day_auc = float(roc_auc_score(y, p)) if 0 < y.sum() < len(y) else float("nan")
91          logger.info("%s: recall=%.3f F1=%.3f", name, result.score.recall, result.score.f1)
92          return result
```

`_evaluate` — проверка одной модели.

- Строки 67–68: результат и списки для расчёта ROC-AUC.
- Строки 69–71: для каждого фолда: проверочные скважины — текущая часть, обучающие — все остальные. `set` — для быстрой проверки «входит ли».
- Строка 72: **новая** модель из фабрики на каждый фолд — знания из прошлого фолда не протекают.
- Строки 73–74: `ModelResult` создаётся при первом фолде, когда стало известно название модели.
- Строки 75–77: обучение с замером времени (`perf_counter` — точный таймер).

- Строки 79–86: для каждой проверочной скважины: вероятности по дням → сохраняем вместе с правильными ответами (для AUC) → декодируем в мероприятия → добавляем метрики скважины к общему `score` → сохраняем строки для Excel.
- Строка 87: сообщение о готовом фолде.
- Строки 89–90: ROC-AUC по всем дням всех проверочных скважин. Условие `0 < y.sum() < len(y)` — AUC определён, только если есть оба класса.
- Строка 91: итог модели в лог.

```
 94      def _rows(self, well: WellSeries, predicted: Sequence) -> list[dict]:
 95          rows = []
 96          for label, hit in self.matcher.match(predicted, well.labels):
 97              rows.append(
 98                  {
 99                      "скважина": well.number,
100                      "куст": well.cluster,
101                      "начало (разметка)": label.start,
102                      "окончание (разметка)": label.end,
103                      "начало (модель)": hit.start if hit else None,
104                      "окончание (модель)": hit.end if hit else None,
105                      "Δ начала, дн": (hit.start - label.start).days if hit else None,
106                      "Δ окончания, дн": (hit.end - label.end).days if hit and hit.end and label.end else
None,
107                  }
108              )
109          return rows
```

`_rows` — строки для Excel: по каждой метке разметки скважина, куст, даты по разметке, даты модели и расхождения в днях. Если модель мероприятие не нашла, её поля пустые (`None`). Расхождение окончания — только если окончание есть и у модели, и в разметке.

evaluation/report.py — отчёт: Excel и график

Файл gtm_detector/ml_freq/evaluation/report.py

Оформляет результаты сравнения: таблица в консоль, книга Excel и картинка с графиками.

```
 1  """Отчёт сравнения моделей: Excel-таблица и график."""
 2
 3  from __future__ import annotations
 4
 5  import logging
 6  from collections.abc import Sequence
 7  from pathlib import Path
 8
 9  import pandas as pd
10
11  from .experiment import ModelResult
12
13  logger = logging.getLogger(__name__)
14
15  BLUE = "#2a78d6"
16  TEXT_SECONDARY = "#6b6a65"
17  GRID = "#e4e3de"
```

Импорты и три цвета: синий для столбиков (первый цвет проверенной палитры), серый для подписей, светлый для сетки.

```
20  class ComparisonReport:
21      """Пишет ``comparison.xlsx`` (метрики + предсказания по моделям) и ``comparison.png``."""
22
23      KEY_METRICS = (
24          "найдено размеченных (recall)",
25          "начало: точно",
26          "начало: ±3 дн",
27          "конец: точно",
28          "конец: ±7 дн",
29          "ROC-AUC по дням",
30      )
31
32      def __init__(self, results: Sequence[ModelResult]) -> None:
33          self.results = list(results)
```

Класс `ComparisonReport`. `KEY_METRICS` — шесть главных метрик для графика и консоли: полнота, точное начало, начало ±3 дня, точное окончание, окончание ±7 дней, ROC-AUC. Конструктор запоминает список результатов моделей.

```
35      def table(self) -> pd.DataFrame:
36          table = pd.DataFrame([result.as_row() for result in self.results]).set_index("модель")
37          return table.sort_values("F1", ascending=False)
38
39      def to_text(self) -> str:
40          table = self.table()[list(self.KEY_METRICS) + ["F1"]]
41          return table.to_string(float_format=lambda value: f"{value:.2f}")
```

- `table()` — таблица pandas: строка на модель, столбцы — все метрики. Индекс — название модели; строки отсортированы по F1 от лучшей к худшей.
- `to_text()` — только ключевые метрики и F1, в виде текста для консоли; числа с двумя знаками после запятой.

```
43      def save(self, out_dir: Path) -> None:
44          out_dir.mkdir(parents=True, exist_ok=True)
45          with pd.ExcelWriter(out_dir / "comparison.xlsx") as writer:
46              self.table().to_excel(writer, sheet_name="Сравнение")
47              for result in self.results:
48                  pd.DataFrame(result.predictions).to_excel(writer, sheet_name=result.name[:31], index=False)
49          self._plot(out_dir / "comparison.png")
50          logger.info("Отчёт: %s", out_dir)
```

`save(out_dir)` — создаёт папку и пишет `comparison.xlsx`: первый лист «Сравнение» — таблица метрик, дальше по листу на каждую модель с её предсказаниями (`[:31]` — Excel не допускает имена листов длиннее 31 символа). Затем рисует `comparison.png`.

```
52      def _plot(self, path: Path) -> None:
53          import matplotlib
54
55          matplotlib.use("Agg")
56          import matplotlib.pyplot as plt
57
58          table = self.table()
59          figure, axes = plt.subplots(2, 3, figsize=(15, 7.5), sharey=True)
60          for axis, metric in zip(axes.flat, self.KEY_METRICS, strict=True):
61              values = table[metric].fillna(0)
62              bars = axis.barh(table.index, values, color=BLUE, height=0.6)
63              for bar, value in zip(bars, values, strict=True):
64                  axis.text(
65                      value + 0.01,
66                      bar.get_y() + bar.get_height() / 2,
67                      f"{value:.0%}",
68                      va="center",
69                      fontsize=9,
70                      color=TEXT_SECONDARY,
71                  )
72              axis.set_xlim(0, 1.12)
73              axis.set_title(metric, loc="left", fontsize=10)
74              axis.invert_yaxis()
75              for side in ("top", "right"):
76                  axis.spines[side].set_visible(False)
77              axis.grid(axis="x", color=GRID)
78              axis.set_axisbelow(True)
79          figure.suptitle("Сравнение моделей (перекрёстная проверка по скважинам)", x=0.01, ha="left")
80          figure.tight_layout()
81          figure.savefig(path, dpi=110)
82          plt.close(figure)
```

`_plot(path)` — график.

- Строки 53–56: matplotlib импортируется **внутри** метода: нужен только для картинки, и без него остальной модуль работает. `matplotlib.use("Agg")` — рисовать в файл без окна (важно на сервере и в фоне).
- Строка 59: сетка 2 × 3 графиков; `sharey=True` — общая ось названий моделей.
- Строки 60–62: для каждой метрики — горизонтальные столбики по моделям (`barh`). Пустые значения → 0.
- Строки 63–71: справа от каждого столбика подпись значения в процентах (`:.0%` превращает 0.69 в «69%»).

- Строки 72–78: оформление: ось от 0 до 1.12 (место для подписей), заголовок слева, лучшая модель сверху (`invert_yaxis`), без верхней и правой рамки, светлая вертикальная сетка позади столбиков.
- Строки 79–82: общий заголовок, плотная компоновка, сохранение (110 точек на дюйм) и закрытие фигуры — чтобы не копилась память.

Файл gtm_detector/ml_freq/evaluation/__init__.py

```
"""Декодирование вероятностей, метрики, перекрёстная проверка и отчёт."""

from .decoding import SegmentDecoder
from .experiment import CrossValidator, ModelResult
from .metrics import EventMatcher, Score
from .report import ComparisonReport

__all__ = ["ComparisonReport", "CrossValidator", "EventMatcher", "ModelResult", "Score", "SegmentDecoder"]
```

Собирает публичные классы подпакета: `SegmentDecoder`, `CrossValidator`, `ModelResult`, `EventMatcher`, `Score`, `ComparisonReport`. Снаружи: `from ..evaluation import CrossValidator`.

cli.py — командная строка

Файл gtm_detector/ml_freq/cli.py

Связывает всё вместе в три команды: `compare` (сравнить модели), `train` (обучить и сохранить), `predict` (найти мероприятия без разметки).

```
"""Командная строка ML-детектора.

Из корня проекта::

    python -m gtm_detector.ml_freq compare                      # сравнить все модели
    python -m gtm_detector.ml_freq compare --models transformer gboost
    python -m gtm_detector.ml_freq train --model transformer    # обучить на всех данных, сохранить веса
    python -m gtm_detector.ml_freq predict --model transformer  # найти мероприятия (разметка не нужна)
"""
```

Docstring с примерами запуска: эти же строки — подсказка для пользователя.

```
from __future__ import annotations

import argparse
import logging
import sys
from collections.abc import Sequence
from datetime import datetime
from pathlib import Path

import pandas as pd

from .config import DataConfig, ExperimentConfig, TrainingConfig
from .data import WellRepository
from .evaluation import ComparisonReport, CrossValidator, SegmentDecoder
from .models import TransformerSegmenter, load_model, model_registry

logger = logging.getLogger(__package__)
```

Импорты: `argparse` — стандартный разбор аргументов командной строки; `datetime` — имя папки по текущему времени; `pandas` — запись результата в Excel; свои модули.

`logging.getLogger(__package__)` — логгер с именем пакета `gtm_detector.ml_freq`.

```
def _run_dir(base: Path) -> Path:
    return base / datetime.now().strftime("%d-%m-%Y_%H-%M-%S")
```

`_run_dir(base)` — папка для результатов текущего запуска: `base/ДД-ММ-ГГГГ_ЧЧ-ММ-СС`. Каждый запуск пишет в свою папку и не затирает прошлые.

```
def _registry(args: argparse.Namespace) -> dict:
    registry = model_registry(seed=args.seed)
    if args.epochs:
        registry["transformer"] = lambda: TransformerSegmenter(
            training=TrainingConfig(epochs=args.epochs, seed=args.seed)
        )
    return registry
```

`_registry(args)` — реестр моделей с учётом аргументов. Если указан `--epochs`, фабрика Transformer заменяется на лямбду, которая создаёт его с нужным числом эпох и тем же зерном. Остальные модели берутся из `model_registry` как есть.

```
43  def run_compare(args: argparse.Namespace) -> int:
44      data = DataConfig(measurements=args.measurements, results=args.results)
45      wells = [well for well in WellRepository(data.measurements, data.results).load() if well.labels]
46      registry = _registry(args)
47      unknown = set(args.models) - set(registry)
48      if unknown:
49          logger.error("Неизвестные модели: %s. Доступны: %s", sorted(unknown), sorted(registry))
50          return 1
51      print(
52          f"Размеченных скважин: {len(wells)}, мероприятий: {sum(len(w.labels) for w in wells)}; "
53          f"модели: {', '.join(args.models)}; фолдов: {args.folds}"
54      )
55
56      validator = CrossValidator(ExperimentConfig(folds=args.folds, seed=args.seed))
57      results = validator.run(wells, {name: registry[name] for name in args.models})
58      report = ComparisonReport(results)
59      out_dir = args.out_dir or _run_dir(data.output_dir)
60      report.save(out_dir)
61      print("\n" + report.to_text())
62      print(f"\nОтчёт: {out_dir / 'comparison.xlsx'}, график: {out_dir / 'comparison.png'}")
63      return 0
```

`run_compare` — команда `compare`.

- Строки 44–45: настройки данных из аргументов и загрузка скважин; остаются только размеченные (`if well.labels`) — без разметки модели не проверить.
- Строки 46–50: проверка, что все запрошенные модели существуют; иначе понятная ошибка со списком доступных и код возврата 1.
- Строки 51–54: краткая сводка: сколько скважин, мероприятий, какие модели и фолды.
- Строки 56–57: перекрёстная проверка выбранных моделей.
- Строки 58–62: отчёт: папка (из `--out-dir` или новая по времени), сохранение Excel и графика, таблица в консоль, пути к файлам.
- Строка 63: код 0 — успех.

```
66  def run_train(args: argparse.Namespace) -> int:
67      data = DataConfig(measurements=args.measurements, results=args.results)
68      wells = [well for well in WellRepository(data.measurements, data.results).load() if well.labels]
69      model = _registry(args)[args.model]()
70      model.fit(wells)
71      path = data.weights_dir / f"{model.name}{model.file_suffix}"
72      model.save(path)
73      print(f"{model.title}: обучена на {len(wells)} скважинах -> {path}")
74      return 0
```

`run_train` — команда `train`. Загружает размеченные скважины, создаёт модель по имени из `--model`, обучает на **всех** скважинах (без перекрёстной проверки — это уже финальная модель) и сохраняет в `weights/<имя><расширение>`, например `weights/gboost.joblib` или `weights/transformer.pt`.

```
 77  def run_predict(args: argparse.Namespace) -> int:
 78      data = DataConfig(measurements=args.measurements)
 79      path = args.weights or _find_weights(data.weights_dir, args.model)
 80      model = load_model(path)
 81      wells = WellRepository(data.measurements, None).load()
 82      decoder = SegmentDecoder()
 83
 84      rows = []
 85      for well in wells:
 86          for interval in decoder.decode(model.predict_proba(well), well.dates):
 87              rows.append(
 88                  {
 89                      "скважина": well.number,
 90                      "куст": well.cluster,
 91                      "начало эффекта": interval.start,
 92                      "окончание эффекта": interval.end,
 93                      "статус": "эффект продолжается" if interval.end is None else "",
 94                  }
 95              )
 96      out_dir = args.out_dir or _run_dir(data.output_dir)
 97      out_dir.mkdir(parents=True, exist_ok=True)
 98      out = out_dir / f"events_{model.name}.xlsx"
 99      pd.DataFrame(rows).to_excel(out, index=False)
100      print(
```

```
101         f"{model.title}: найдено {len(rows)} мероприятий на "
102         f"{len({(r['скважина'], r['куст']) for r in rows})} скважинах -> {out}"
103     )
104     return 0
```

`run_predict` — команда `predict`: поиск мероприятий без разметки.

- Строка 78: только файл замеров — разметка не нужна.
- Строки 79–80: путь к весам — из `--weights` или найти в папке весов по имени модели; загрузка (`load_model` сам понимает тип модели по расширению).
- Строка 81: все скважины (без разметки: `None`).
- Строки 84–95: для каждой скважины: вероятности → декодер → по строке на каждое найденное мероприятие: скважина, куст, начало, окончание и статус «эффект продолжается», если окончания нет.
- Строки 96–99: запись в `events_<модель>.xlsx` в папке результата.
- Строки 100–103: итог: сколько мероприятий и на скольких скважинах (множество пар номер/куст считает уникальные скважины).

```
107 def _find_weights(directory: Path, name: str) -> Path:
108     for suffix in (".pt", ".joblib"):
109         candidate = directory / f"{name}{suffix}"
110         if candidate.exists():
111             return candidate
112     raise FileNotFoundError(f"Нет весов модели {name} в {directory}: сначала выполните train")
```

`_find_weights` — ищет файл весов модели: сначала `.pt`, потом `.joblib`. Не нашёл — `FileNotFoundError` с подсказкой «сначала выполните train».

```
115 def build_parser(prog: str | None = None) -> argparse.ArgumentParser:
116     defaults = DataConfig()
117     names = list(model_registry())
118     parser = argparse.ArgumentParser(
119         prog=prog or f"python -m {__package__}",
120         description="ML-детекция мероприятий «увеличение частоты F»: Transformer и модели scikit-learn.",
121     )
122     parser.add_argument("-v", "--verbose", action="store_true", help="подробный лог")
123     commands = parser.add_subparsers(dest="command", required=True)
124
125     def common(sub: argparse.ArgumentParser, with_results: bool = True) -> None:
126         sub.add_argument("--measurements", type=Path, default=defaults.measurements, help="файл замеров")
127         if with_results:
128             sub.add_argument("--results", type=Path, default=defaults.results, help="файл с разметкой")
129         sub.add_argument("--seed", type=int, default=0, help="зерно случайности")
130         sub.add_argument("--epochs", type=int, help="эпох обучения Transformer (по умолчанию 80)")
131
```

`build_parser` — описание всех команд и ключей.

- Строки 116–117: значения по умолчанию (пути из `DataConfig`) и список имён моделей.
- Строки 118–121: главный парсер; `prog` — как программа называется в подсказке.
- Строка 122: общий ключ `-v` / `--verbose` — подробный лог.
- Строка 123: подкоманды; `required=True` — без команды запуск невозможен.
- Строки 125–130: функция `common` добавляет общие ключи к подкоманде: `--measurements`, `--results` (кроме `predict`), `--seed`, `--epochs`. Так одни и те же ключи не описываются трижды.

```
132     compare = commands.add_parser("compare", help="перекрёстная проверка и сравнение моделей")
133     common(compare)
134     compare.add_argument("--models", nargs="+", default=names, help=f"какие модели: {' '.join(names)}")
135     compare.add_argument("--folds", type=int, default=5, help="фолдов по скважинам")
136     compare.add_argument(
137         "--out-dir", type=Path, help="папка отчёта (по умолчанию results/ml_freq/<дата_время>)"
138     )
139     compare.set_defaults(handler=run_compare)
140
141     train = commands.add_parser("train", help="обучить модель на всей разметке и сохранить веса")
142     common(train)
143     train.add_argument("--model", choices=names, default="transformer")
144     train.set_defaults(handler=run_train)
145
146     predict = commands.add_parser("predict", help="найти мероприятия обученной моделью")
147     common(predict, with_results=False)
148     predict.add_argument("--model", choices=names, default="transformer")
149     predict.add_argument("--weights", type=Path, help="файл весов (по умолчанию из ml_freq/weights)")
150     predict.add_argument("--out-dir", type=Path, help="папка результата")
151     predict.set_defaults(handler=run_predict)
152     return parser
```

Три подкоманды:

- `compare`: общие ключи + `--models` (`nargs="+"` — один или несколько, по умолчанию все), `--folds`, `--out-dir`;
- `train`: общие ключи + `--model` (`choices` — только из списка известных моделей; по умолчанию transformer);
- `predict`: общие ключи без `--results` + `--model`, `--weights`, `--out-dir`.

`set_defaults(handler=run_...)` привязывает к подкоманде функцию, которая её выполняет. После разбора аргументов достаточно вызвать `args.handler(args)` — без цепочки `if command == ...`.

```
155 def main(argv: Sequence[str] | None = None, prog: str | None = None) -> int:
156     args = build_parser(prog).parse_args(argv)
157     logging.basicConfig(
158         level=logging.INFO if args.verbose else logging.WARNING,
159         format="%(asctime)s %(levelname)s %(name)s: %(message)s",
160     )
161     logging.getLogger(f"{__package__}.evaluation").setLevel(logging.INFO)
162     return args.handler(args)
163
164
165 if __name__ == "__main__":
166     sys.exit(main())
```

`main(argv, prog)` — точка входа.

- Строка 156: разбор аргументов. `argv=None` — взять из командной строки; передать список можно в тестах или из другого скрипта.
- Строки 157–160: настройка логов: с `-v` — подробные (INFO), иначе только предупреждения и ошибки.
- Строка 161: сообщения модуля `evaluation` (о готовых фолдах) показываются всегда — сравнение длится минуты, и видно, что программа не зависла.
- Строка 162: вызов функции выбранной команды; её код возврата — код возврата программы.
- Строки 165–166: позволяет запускать и сам файл напрямую (`python cli.py`), хотя обычно используется `python -m gtm_detector.ml_freq`.

tests/conftest.py — синтетические скважины для тестов

Файл gtm_detector/ml_freq/tests/conftest.py

`conftest.py` — специальный файл pytest: фикстуры из него автоматически доступны всем тестам папки. Здесь — генератор искусственной скважины с заранее известными датами мероприятия: тест знает правильный ответ и не зависит от больших Excel-файлов.

```
"""Синтетические скважины: подъём частоты и отклик добычи с известными датами."""

from __future__ import annotations

from datetime import date, timedelta

import numpy as np
import pandas as pd
import pytest

from gtm_detector.ml_freq.data import Interval, WellSeries

START = date(2026, 1, 1)
```

Импорты: `timedelta` — сдвиг даты на N дней; NumPy, pandas; `pytest` — для декоратора фикстуры; `Interval`, `WellSeries` — то, что создаём.

`START = date(2026, 1, 1)` — первый день всех синтетических рядов.

```
def day(offset: int) -> date:
    return START + timedelta(days=offset)
```

`day(offset)` — дата через `offset` дней после 01.01.2026. `day(0)` — 01.01, `day(42)` — 12.02. Тесты пишут номера дней, а не даты — так нагляднее.

```
def make_well(n_days: int = 120, step: int | None = 40, effect: int = 30, seed: int = 0) -> WellSeries:
    """Частота 48 -> 50 Гц в день ``step``, Qж/Qн выше базы ``effect`` дней, затем возврат."""
    rng = np.random.default_rng(seed)
    index = pd.date_range(START, periods=n_days, freq="D")
    frequency = np.full(n_days, 48.0)
    liquid = np.full(n_days, 100.0)
    labels = []
    if step is not None:
        frequency[step:] = 50.0
        liquid[step + 2 : step + 2 + effect] = 110.0
        labels.append(Interval(day(step + 2), day(step + 1 + effect), day(step)))
```

`make_well(n_days, step, effect, seed)` — создаёт скважину.

- Строка 22: генератор случайных чисел с зерном — шум одинаковый при каждом запуске.
- Строка 23: календарь из `n_days` дней подряд.
- Строки 24–25: частота везде 48 Гц, дебит жидкости везде 100.
- Строки 27–30: если задан `step` (номер дня подъёма частоты): частота с этого дня 50 Гц; дебит с дня `step+2` на `effect` дней — 110 (отклик через 2 дня, как в реальности); в разметку добавляется мероприятие с началом `day(step+2)`, окончанием `day(step+1+effect)` (последний день повышенного дебита) и датой выполнения `day(step)`.

При `step=None` получается скважина без мероприятия.

```
    frame = pd.DataFrame(
        {
            "frequency": frequency,
            "debit_liquid": liquid + rng.normal(0, 0.3, n_days),
            "debit_oil": liquid * 0.1,
            "watercut": np.full(n_days, 90.0),
            "pressure_zab": np.full(n_days, 50.0),
            "pressure_line": np.full(n_days, 12.0),
            "state": np.ones(n_days),
        },
        index=index,
    )
    return WellSeries(number=f"W{seed}", cluster="1", frame=frame, labels=labels)
```

Таблица рядов скважины:

- дебит жидкости — с небольшим шумом (±0.3), чтобы ряд не был идеально гладким;
- дебит нефти — 10% от жидкости;
- обводнённость 90%, давления постоянные, скважина работает все дни (`state = 1`).

Возвращается `WellSeries` с номером `W<seed>` и кустом «1».

```
@pytest.fixture
def wells() -> list[WellSeries]:
    return [make_well(step=30 + 3 * i, effect=25 + 2 * i, seed=i) for i in range(8)]
```

Фикстура `wells`. `@pytest.fixture` — если тест принимает аргумент с именем `wells`, pytest сам вызовет эту функцию и передаст результат. Создаёт 8 разных скважин: подъём частоты в дни 30, 33, 36…, эффект от 25 до 39 дней. Разнообразие нужно, чтобы модели в тестах учились на разных примерах.

tests/test_models_and_evaluation.py — тесты моделей и оценки

Файл gtm_detector/ml_freq/tests/test_models_and_evaluation.py

Проверяет декодер, сопоставление с разметкой, обе модели (обучение, предсказание, сохранение и загрузка) и перекрёстную проверку целиком. Модели в тестах уменьшены, чтобы всё выполнялось за секунды.

```
1  import numpy as np
2  import torch
3
4  from gtm_detector.ml_freq.config import DecoderConfig, ExperimentConfig, TrainingConfig, TransformerConfig
5  from gtm_detector.ml_freq.data import Interval
6  from gtm_detector.ml_freq.evaluation import CrossValidator, EventMatcher, SegmentDecoder
7  from gtm_detector.ml_freq.models import (
8      SklearnSegmenter,
9      TimeSeriesTransformer,
10     TransformerSegmenter,
11     classic_models,
12     load_model,
13 )
14 from gtm_detector.ml_freq.tests.conftest import day, make_well
```

Импорты проверяемых классов и генератора синтетических скважин. Скобки в строках 7–13 — перенос длинного импорта на несколько строк по PEP 8.

```
16 SMALL = TransformerConfig(d_model=16, n_heads=2, n_layers=1, dim_feedforward=32)
17 QUICK = TrainingConfig(epochs=3, crop_length=64)
```

Две тестовые конфигурации:

- `SMALL` — крошечная сеть: d_model 16, 2 головы, 1 слой;
- `QUICK` — 3 эпохи на окнах по 64 дня.

Тесты проверяют, что код **работает**, а не качество модели, поэтому сеть маленькая и обучение короткое.

```
20 def test_decoder_merges_gaps_and_marks_open_end():
21     proba = np.zeros(40)
22     proba[5:15] = 0.9
23     proba[16:20] = 0.9  # разрыв в 1 день — склеится
24     proba[35:] = 0.9  # доходит до конца — эффект продолжается
25     decoder = SegmentDecoder(DecoderConfig(smooth_days=1, max_gap=2, min_length=2))
26     events = decoder.decode(proba, [day(i) for i in range(40)])
27     assert events == [Interval(day(5), day(19)), Interval(day(35), None)]
```

`test_decoder_merges_gaps_and_marks_open_end` — декодер на заранее заданных вероятностях:

- дни 5–14 и 16–19 по 0.9 с разрывом в один день (15-й) → должны склеиться в один период 5–19;
- дни 35–39 (до конца ряда) по 0.9 → период с открытым окончанием (`None`).

Сглаживание выключено (`smooth_days=1`), чтобы границы были точно предсказуемы. Проверяется точное совпадение списка мероприятий.

```
30 def test_matcher_one_to_one_by_start():
31     labels = [Interval(day(10), day(20)), Interval(day(60), day(70))]
32     predicted = [Interval(day(11), day(20)), Interval(day(12), day(25)), Interval(day(100), None)]
33     score = EventMatcher(start_tolerance=7).score(predicted, labels)
34     assert (score.true_positives, score.false_negatives, score.false_positives) == (1, 1, 2)
35     assert score.start_errors == [1] and score.end_errors == [0]
```

`test_matcher_one_to_one_by_start` — сопоставление:

- в разметке 2 мероприятия (начала 10 и 60), модель нашла 3 (начала 11, 12 и 100);
- метку «10» должна забрать ближайшая находка «11» (разница 1), находка «12» — лишняя, хотя тоже в пределах 7 дней (один-к-одному);
- метку «60» никто не нашёл (FN), находка «100» — лишняя.

Итог: TP=1, FN=1, FP=2; ошибка начала +1 день, ошибка окончания 0 (обе заканчиваются 20-го).

```
38  def test_transformer_forward_shape():
39      network = TimeSeriesTransformer(n_channels=5, config=SMALL)
40      logits = network(torch.randn(2, 30, 5))
41      assert logits.shape == (2, 30)
```

`test_transformer_forward_shape` — форма выхода сети: на вход случайный тензор (2 скважины × 30 дней × 5 признаков), на выход должно быть 2 × 30 — по числу на день. Ловит ошибки перестановки осей.

```
44  def test_transformer_fit_predict_save_load(tmp_path, wells):
45      model = TransformerSegmenter(SMALL, QUICK).fit(wells)
46      proba = model.predict_proba(wells[0])
47      assert proba.shape == (len(wells[0]),) and ((proba >= 0) & (proba <= 1)).all()
48      path = tmp_path / "transformer.pt"
49      model.save(path)
50      assert np.allclose(load_model(path).predict_proba(wells[0]), proba, atol=1e-6)
```

`test_transformer_fit_predict_save_load` — полный цикл Transformer:

- обучение на 8 синтетических скважинах;
- предсказание: длина равна числу дней, все значения от 0 до 1;
- сохранение в файл во временной папке (`tmp_path` — встроенная фикстура pytest, папка удаляется после теста);
- загрузка и повторное предсказание должны совпасть с исходным с точностью до 0.000001 (`np.allclose`). Это гарантирует, что сохраняется всё нужное: архитектура, веса, признаки.

```
53  def test_sklearn_models_learn_the_effect(tmp_path, wells):
54      model = classic_models()["gboost"]().fit(wells)
55      test = make_well(step=50, effect=30, seed=99)
56      proba = model.predict_proba(test)
57      assert proba[60:75].mean() > 0.7 and proba[:40].mean() < 0.3
58      path = tmp_path / "gboost.joblib"
59      model.save(path)
60      assert isinstance(load_model(path), SklearnSegmenter)
```

`test_sklearn_models_learn_the_effect` — проверка, что модель действительно **учится**:

- градиентный бустинг обучается на 8 скважинах;
- проверка на новой скважине (подъём в 50-й день, эффект 52–81);
- в середине эффекта (дни 60–74) средняя вероятность должна быть выше 0.7, задолго до подъёма (дни 0–39) — ниже 0.3;
- сохранение и загрузка дают объект нужного класса.

```
63  def test_cross_validation_runs_for_all_models(wells):
64      models = {"logreg": classic_models()["logreg"], "transformer": lambda: TransformerSegmenter(SMALL,
QUICK)}
65      results = CrossValidator(ExperimentConfig(folds=2)).run(wells, models)
66      assert [r.name for r in results] == ["logreg", "transformer"]
67      for result in results:
68          assert result.score.true_positives + result.score.false_negatives == len(wells)
69          assert 0.0 <= result.day_auc <= 1.0
```

`test_cross_validation_runs_for_all_models` — перекрёстная проверка от начала до конца на двух моделях (логистическая регрессия и маленький Transformer), 2 фолда:

- результаты в том же порядке, что и модели;
- каждая размеченная скважина учтена ровно один раз: TP + FN = 8;
- ROC-AUC — корректное число от 0 до 1.